1. Introduction

This document defines a complete technical architecture for a secure, hardware-bound Local Copilot API for Windows 11.
The purpose of this system is to enable Microsoft Copilot to interact with the local operating system through a controlled, sandboxed, and auditable interface. The API allows Copilot to perform deterministic system tasks, automate workflows, and manipulate local resources without relying on cloud execution.

The design addresses the following core requirements:

- **Local execution only:** All operations occur on the user's device. No cloud involvement.
- **Strict security boundaries:** The API is only available when Windows 11 security modules are fully active.
- **User-controlled automation:** Copilot executes actions only after explicit user confirmation.
- **Deterministic behavior:** The API exposes only predefined, safe operations.
- **Auditability:** All actions are logged in a tamper-resistant, TPM-bound audit log.
- **EU digital sovereignty compliance:** No data leaves the device; all processing is local.

This document is written as a technical specification.
It contains no marketing language, no promotional framing, and no PR-style formulations.
It is intended for engineers, architects, and security analysts.

---

2. Problem Statement

Microsoft Copilot currently operates as a conversational system without the ability to interact with the local Windows environment.
This creates a structural limitation: Copilot can generate information, but it cannot execute or automate real tasks on the system.

The following gaps exist:

2.1 No Local Workflow Execution

Copilot cannot automate repetitive or structured tasks such as:

- organizing files
- converting media
- extracting logs
- batch renaming
- preparing project structures

This prevents Copilot from functioning as a practical automation tool.

2.2 No System Interaction

Copilot cannot:

- launch applications
- close applications
- switch window focus
- trigger system utilities
- manage clipboard content

This isolates Copilot from the operating system.

2.3 No File Manipulation

Copilot cannot:

- create files
- modify files
- read local data
- write structured output to disk

This prevents integration with real workflows.

2.4 No Enterprise Integration

Enterprises cannot use Copilot for:

- internal automation
- compliance-bound workflows
- secure local processing

The lack of a controlled execution layer makes Copilot unsuitable for regulated environments.

2.5 No EU-Compliant Local Processing

Many organizations require:

- local-only execution
- no cloud dependency
- full auditability

Copilot cannot meet these requirements in its current form.

Conclusion:
The absence of a secure, hardware-bound local API isolates Copilot from Windows and prevents it from becoming a productive automation component.
A sandboxed, deterministic, auditable local execution interface is required to close this gap.

---

3. Design Principles

The Local Copilot API is based on a set of strict engineering principles that define its behavior, security posture, and operational boundaries.
These principles ensure that the system remains predictable, secure, and compliant with regulatory requirements.

3.1 Local Execution Only

All operations are executed exclusively on the user's device.
No cloud execution, no remote code, and no external dependencies are permitted.

Rationale:

- Eliminates cloud-related attack vectors.
- Ensures compliance with data-sovereignty requirements.
- Guarantees deterministic behavior independent of network conditions.

3.2 Hardware-Bound Security

The API is only available when specific Windows 11 security modules are active.
This ensures that the execution environment is protected against kernel-level, memory-level, and boot-level attacks.

Rationale:

- Prevents API activation on insecure systems.
- Ensures that the sandbox cannot be bypassed.
- Provides a verifiable hardware root of trust.

3.3 Explicit User Control

Copilot never executes actions autonomously.
Every operation requires explicit user confirmation through a permission gateway.

Rationale:

- Prevents unauthorized automation.
- Ensures user awareness of all system-level actions.
- Eliminates the risk of silent or hidden operations.

3.4 Minimal Attack Surface

The API exposes only predefined, deterministic operations.
No arbitrary code execution, no script execution, and no unrestricted system calls are allowed.

Rationale:

- Reduces the risk of exploitation.
- Ensures predictable behavior.
- Prevents misuse by malicious prompts or manipulated input.

3.5 Deterministic and Auditable

All actions are logged in a TPM-bound audit log.
Logs cannot be modified or deleted without breaking integrity.

Rationale:

- Enables forensic analysis.
- Ensures accountability.
- Supports enterprise compliance requirements.

3.6 EU Digital Sovereignty Compliance

The architecture ensures:

- no data leaves the device
- no cloud dependency
- full user control
- transparent auditability

Rationale:

- Meets EU regulatory expectations.
- Supports public-sector and enterprise adoption.
- Aligns with sovereignty-focused digital strategies.

Summary:
These principles define the operational boundaries of the Local Copilot API and ensure that the system remains secure, predictable, and compliant across all usage scenarios.

---

4. Security Requirements

The Local Copilot API is disabled by default.
It becomes available only when the system meets a strict set of hardware-bound and OS-level security conditions.
These requirements ensure that the execution environment cannot be exploited, bypassed, or manipulated.

4.1 Mandatory Security Modules

The following Windows 11 security components must be active for the API to initialize:

Module	Requirement	Purpose
TPM 2.0	Mandatory	Provides hardware-rooted identity, secure key storage, and integrity measurement.
Secure Boot	Mandatory	Ensures the boot chain is trusted and prevents pre-boot tampering.
Core Isolation	Mandatory	Protects kernel memory and enforces isolation boundaries.
Memory Integrity (HVCI)	Mandatory	Blocks unsigned or malicious kernel-mode drivers.
Virtualization-Based Security (VBS)	Mandatory	Provides an isolated execution environment for the Local Execution Layer.
Smart App Control	Mandatory	Prevents execution of untrusted binaries and scripts.

If any module is inactive, misconfigured, or unsupported →
The Local Copilot API remains fully disabled.

4.2 Hardware Requirements

To ensure isolation and prevent privilege escalation, the following hardware capabilities are required:

- CPU with virtualization extensions (Intel VT-x or AMD-V)
- TPM 2.0 firmware or discrete module
- UEFI firmware with Secure Boot support
- Memory protection extensions (e.g., Mode-based Execution Control)

These requirements ensure that the sandbox cannot be bypassed through hardware-level attacks.

4.3 API Activation Conditions

The API activates only when all of the following conditions are met:

1. All mandatory security modules are active.
2. System integrity checks pass (TPM-measured boot).
3. VBS is running and verified.
4. Smart App Control is in “On” or “Evaluation” mode.
5. No unsigned kernel drivers are loaded.
6. No integrity violations are detected in the Windows Security Baseline.

If any condition fails at runtime →
The API is immediately suspended.

4.4 Isolation Requirements

The Local Execution Layer must run inside a VBS-protected environment with:

- isolated memory regions
- no direct kernel access
- no direct hardware access
- no ability to load external libraries
- no ability to spawn child processes
- no ability to execute arbitrary code

Only predefined, deterministic operations are permitted.

4.5 Permission Model Requirements

Every operation must pass through a permission gateway that enforces:

- explicit user confirmation
- granular access control
- operation-specific restrictions
- time-limited permissions
- revocation at any time

No persistent permission is granted without explicit user approval.

4.6 Audit Requirements

All actions must be logged with:

- timestamp
- operation type
- parameters
- user confirmation state
- execution result
- integrity hash (TPM-bound)

Logs must be:

- local only
- tamper-resistant
- exportable only with user consent

Summary:
The security requirements ensure that the Local Copilot API can only operate in a hardened, verifiably secure environment.
If any security guarantee is weakened, the API automatically deactivates.

---

5. System Architecture Overview

The Local Copilot API is structured as a layered architecture.
Each layer has a clearly defined responsibility and a strict security boundary.
No layer can bypass the layer above or below it.

The architecture ensures:

- deterministic behavior
- strict isolation
- controlled execution
- verifiable security
- minimal attack surface

5.1 Architecture Diagram

```
+-----------------------------------------------+
|                  Copilot Chat                 |
|           (Natural Language Interface)        |
+-----------------------+-----------------------+
                        |
                        v
+-----------------------------------------------+
|            Command Validation Layer           |
|   (Intent Analysis, Syntax Check, Safety Gate)|
+-----------------------+-----------------------+
                        |
                        v
+-----------------------------------------------+
|           Local Execution Layer (LEL)         |
|      (VBS-Isolated, Restricted Operations)    |
+-----------------------+-----------------------+
                        |
                        v
+-----------------------------------------------+
|               Permission Gateway              |
|   (User Confirmation, ACL Enforcement, Policy)|
+-----------------------+-----------------------+
                        |
                        v
+-----------------------------------------------+
|              Windows 11 System APIs           |
|    (File System, App Control, Utilities, Logs)|
+-----------------------------------------------+
```

5.2 Layer Responsibilities (High-Level)

Copilot Chat Layer

- Converts natural language into structured commands.
- Provides human-readable previews.
- Does not execute anything directly.

Command Validation Layer

- Ensures commands are valid, safe, and allowed.
- Rejects anything outside the API surface.
- Prevents arbitrary code execution.

Local Execution Layer (LEL)

- Executes only predefined operations.
- Runs inside a VBS-isolated environment.
- Has no direct kernel or hardware access.

Permission Gateway

- Requires explicit user approval for every action.
- Enforces granular access control.
- Logs all decisions.

Windows 11 System APIs

- Provide the actual system-level functionality.
- Are only accessible through the LEL.
- Cannot be invoked directly by Copilot.

5.3 Data Flow Summary

1. User issues a natural language request.
2. Copilot interprets the request and generates a structured command.
3. The command is validated for safety and correctness.
4. The user receives a preview and must confirm the action.
5. The Local Execution Layer performs the operation in isolation.
6. The result is returned to Copilot.
7. The action is logged in a TPM-bound audit log.

5.4 Isolation Guarantees

- No direct access to kernel APIs.
- No ability to load external libraries.
- No ability to spawn processes.
- No ability to execute scripts or binaries.
- All operations must pass through the permission gateway.
- All operations must be part of the predefined API surface.

Summary:
The architecture ensures that Copilot can interact with Windows in a controlled, secure, and deterministic manner.
Each layer enforces strict boundaries, preventing misuse, escalation, or unauthorized automation.

---

6. Component Descriptions

This section defines each architectural component in detail.
Every component has a strict responsibility and cannot exceed its defined boundaries.
No component may bypass the layer above or below it.

6.1 Copilot Chat Layer

Purpose

The Copilot Chat Layer is the natural-language interface.
It does not execute actions.
It only interprets user intent and generates structured commands.

Responsibilities

- Convert natural language into structured, machine-readable commands.
- Provide a human-readable preview of the intended action.
- Perform initial intent classification.
- Reject ambiguous or unsafe requests before they reach deeper layers.

Restrictions

- Cannot access system APIs.
- Cannot execute any operation.
- Cannot bypass validation or permissions.

6.2 Command Validation Layer

Purpose

This layer ensures that all commands are valid, safe, and within the allowed API surface.

Responsibilities

- Validate syntax and structure of generated commands.
- Verify that the requested operation is part of the approved API surface.
- Perform static safety analysis (no arbitrary code, no scripts, no unbounded operations).
- Reject malformed, unsafe, or unauthorized commands.

Restrictions

- Cannot execute commands.
- Cannot escalate privileges.
- Cannot modify system state.

6.3 Local Execution Layer (LEL)

Purpose

The Local Execution Layer performs the actual system operations.
It runs inside a VBS-isolated environment to prevent privilege escalation or kernel interaction.

Responsibilities

- Execute only predefined, deterministic operations.
- Operate strictly within the VBS sandbox.
- Interact with Windows APIs through restricted, controlled channels.
- Return structured results to the upper layers.

Restrictions

- No direct kernel access.
- No ability to load external libraries.
- No ability to spawn processes.
- No ability to execute scripts, binaries, or arbitrary code.
- No network access unless explicitly permitted by the permission gateway.

6.4 Permission Gateway

Purpose

The Permission Gateway enforces explicit user consent for every operation.

Responsibilities

- Display a preview of the action to the user.
- Require explicit confirmation before execution.
- Enforce granular access control (file, folder, registry, application).
- Apply time-limited or one-time permissions.
- Allow users to revoke permissions at any time.

Restrictions

- Cannot auto-approve actions.
- Cannot store persistent permissions without explicit user approval.
- Cannot bypass user interaction.

6.5 Audit Log

Purpose

The audit log provides a tamper-resistant record of all actions executed through the Local Copilot API.

Responsibilities

- Record every operation with timestamp, parameters, and result.
- Bind log entries to TPM for integrity protection.
- Prevent modification or deletion without breaking integrity.
- Allow export only with explicit user consent.

Restrictions

- Logs remain local only.
- No automatic cloud synchronization.
- No silent logging bypass.

Summary:
Each component has a clearly defined role and strict boundaries.
Together, they form a secure, deterministic, auditable execution pipeline for local Copilot operations.

---

7. Allowed Operations (API Surface)

The Local Copilot API exposes a strictly limited, deterministic set of operations.
No arbitrary code execution, no scripting, and no unrestricted system calls are permitted.
Every operation must be predictable, auditable, and safe.

The API surface is divided into functional categories.

7.1 File System Operations

These operations allow controlled interaction with the Windows file system.

Allowed:

- Create folder
- Create file
- Move file
- Rename file
- Delete file (requires explicit confirmation)
- Read file contents
- Write text to file
- List directory contents
- Query file metadata (size, timestamps, attributes)

Restrictions:

- No raw disk access
- No access outside user-approved paths
- No ability to modify system directories
- No binary execution
- No script execution

7.2 Application Control

These operations allow controlled interaction with installed applications.

Allowed:

- Launch application (from approved paths)
- Close application
- Switch window focus
- Query running processes (read-only)

Restrictions:

- No ability to inject code
- No ability to modify process memory
- No ability to spawn background processes
- No ability to elevate privileges

7.3 System Utilities

These operations expose safe, predefined system utilities.

Allowed:

- Query system information (CPU, RAM, OS version)
- Trigger predefined maintenance tasks (disk cleanup, temp file removal)
- Manage clipboard (optional, user-controlled)
- Query network status (read-only)

Restrictions:

- No registry modification outside approved keys
- No network configuration changes
- No firewall or security settings changes
- No driver interaction

7.4 Workflow Automation

These operations allow Copilot to automate repetitive tasks using built-in Windows capabilities.

Allowed:

- Batch rename files
- Batch convert images using Windows codecs
- Organize files by rules (date, type, size)
- Extract logs from predefined locations
- Filter logs by patterns
- Generate structured output (JSON, CSV, TXT)
- Create project folder structures

Restrictions:

- No custom script execution
- No external tool invocation
- No dynamic code generation

7.5 No Arbitrary Code Execution

The following are explicitly prohibited:

- PowerShell execution
- CMD execution
- WSL execution
- Python, Node.js, or any interpreter execution
- Loading DLLs
- Running EXE files outside Smart App Control whitelist
- Creating or modifying scheduled tasks
- Creating services
- Modifying system startup entries

These restrictions ensure that the API cannot be used to escalate privileges, execute malware, or bypass system protections.

7.6 Operation Validation Rules

Every operation must satisfy:

1. **Is part of the approved API surface**
2. **Has a deterministic outcome**
3. **Requires explicit user confirmation**
4. **Is executed inside the VBS-isolated Local Execution Layer**
5. **Is logged in the TPM-bound audit log**
6. **Cannot modify system integrity**
7. **Cannot escalate privileges**

Summary:
The allowed operations define a minimal, safe, deterministic API surface that enables practical automation while preventing misuse, exploitation, or unauthorized system modification.

---

8. Why Microsoft Gains Immediate Strategic Advantage

This section explains, in strictly technical and structural terms, why Microsoft would gain an immediate and measurable advantage by implementing a Local Copilot API.
No marketing language is used.
The arguments are based on architecture, security posture, ecosystem position, and regulatory alignment.

8.1 Windows Already Provides the Required Security Infrastructure

Windows 11 includes a unified, hardware-anchored security baseline that no other major OS currently enforces at scale:

- TPM 2.0 as a universal requirement
- Secure Boot as a default
- Virtualization-Based Security (VBS) as a mature isolation layer
- Memory Integrity (HVCI) for kernel-level protection
- Smart App Control for binary trust enforcement

Because these components already exist, Microsoft can implement a secure local execution layer **without introducing new attack surfaces**.

This reduces engineering risk and accelerates deployment.

8.2 No Competitor Offers Local AI Automation at OS Level

Other ecosystems lack a unified, secure, hardware-bound automation interface:

- Apple provides local ML but no OS-level automation sandbox with hardware-verified isolation.
- Linux distributions lack a standardized, enforced security baseline across devices.
- Google’s ecosystem relies heavily on cloud-centric models and does not expose local OS automation through a controlled AI interface.

A Local Copilot API would make Windows the **first operating system** with:

- a deterministic AI automation layer
- hardware-verified execution
- OS-integrated permission control
- a sandboxed, auditable execution environment

This establishes a unique technical position.

8.3 Alignment With EU Digital Sovereignty Requirements

The EU increasingly demands:

- local processing
- no cloud dependency
- transparent auditability
- user-controlled data boundaries
- verifiable security guarantees

The Local Copilot API satisfies these requirements by design:

- all operations are local
- all actions are logged
- no data leaves the device
- execution is hardware-isolated
- permissions are explicit and user-controlled

This makes the architecture suitable for:

- public sector
- regulated industries
- critical infrastructure
- enterprises with strict compliance requirements

Microsoft becomes the only major vendor with a compliant, AI-enabled local automation layer.

8.4 Enterprise Adoption Benefits

Enterprises avoid cloud-executed automation due to:

- compliance restrictions
- data protection requirements
- audit obligations
- risk of remote code execution
- lack of deterministic behavior

A local, hardware-secured Copilot API resolves these issues:

- deterministic execution
- no external dependencies
- full audit trail
- no cloud exposure
- predictable security posture

This enables enterprise-grade automation without violating internal policies.

8.5 Expansion of the Developer Ecosystem

A stable, restricted API surface allows developers to build:

- workflow modules
- automation extensions
- domain-specific tools
- enterprise integrations

Because the API is deterministic and sandboxed, developers can rely on:

- predictable behavior
- stable interfaces
- controlled permissions
- safe execution boundaries

This creates a new ecosystem layer without introducing security risks.

8.6 Strategic Summary

Implementing the Local Copilot API gives Microsoft:

- the first OS-level AI automation layer
- a hardware-secured execution environment unmatched by competitors
- compliance alignment with EU sovereignty requirements
- a strong position in enterprise and public-sector markets
- a foundation for a new developer ecosystem
- minimal engineering risk due to existing Windows security infrastructure

These advantages arise directly from the architecture and security model, not from marketing or positioning.

---

9. Threat Model

The threat model defines which attack vectors the Local Copilot API must defend against, which threats are inherently mitigated by the architecture, and which residual risks remain. The model is based on realistic adversary capabilities and the constraints of Windows 11's security infrastructure.

9.1 Threat Categories

The system must defend against the following categories of threats:

1. **Remote attackers** attempting to exploit Copilot to execute malicious code.
2. **Local attackers** attempting to escalate privileges or bypass system protections.
3. **Malicious prompts** attempting to trick Copilot into unsafe operations.
4. **Compromised applications** attempting to misuse the API.
5. **Misconfigurations** that weaken system integrity.
6. **Unauthorized automation** triggered without user consent.

The architecture is designed to neutralize these threats through isolation, validation, permissions, and hardware-bound security.

9.2 Prevented Threats

The following threats are fully mitigated by design:

9.2.1 Remote Code Execution via Copilot

- No arbitrary code execution is allowed.
- No script execution (PowerShell, CMD, WSL, Python, Node.js).
- No binary execution outside Smart App Control whitelist.
- All operations must be part of the predefined API surface.

9.2.2 Local Privilege Escalation

- The Local Execution Layer runs inside VBS.
- No direct kernel access.
- No ability to load drivers or DLLs.
- No ability to spawn processes.

9.2.3 Kernel Manipulation

- Memory Integrity (HVCI) blocks unsigned kernel drivers.
- Secure Boot ensures trusted boot chain.
- VBS isolates execution from kernel memory.

9.2.4 Unauthorized Automation

- Every operation requires explicit user confirmation.
- No persistent permissions without user approval.
- No silent execution paths.

9.2.5 Data Exfiltration

- No network access unless explicitly approved.
- No cloud synchronization of logs or actions.
- All operations are local.

9.2.6 Tampering with Logs

- Audit logs are TPM-bound.
- Integrity hashes prevent modification.
- Logs cannot be silently deleted.

9.3 Partially Mitigated Threats

These threats cannot be fully eliminated but are significantly reduced:

9.3.1 User Approves Malicious Actions

Residual risk:
A user may intentionally or accidentally approve a harmful operation.

Mitigations:

- Clear previews of actions.
- Granular permission prompts.
- Warnings for destructive operations.
- Audit logs for forensic review.

9.3.2 Misconfigured System Security

Residual risk:
Users may disable required security modules.

Mitigations:

- API deactivates automatically.
- System warns user that Copilot local execution is unavailable.
- No fallback mode.

9.3.3 Physical Access Attacks

Residual risk:
An attacker with physical access may attempt to manipulate the system.

Mitigations:

- TPM-bound logs.
- Secure Boot.
- VBS isolation.
- No persistent permissions without explicit approval.

9.4 Out-of-Scope Threats

The following threats are outside the scope of the Local Copilot API:

- Hardware implants
- Firmware-level attacks
- Supply-chain compromises
- Attacks requiring physical device theft combined with advanced hardware tampering
- Compromise of third-party applications outside the API boundary

These threats are addressed by broader Windows security measures, not by the API itself.

9.5 Threat Model Summary

The Local Copilot API:

- prevents remote code execution
- prevents privilege escalation
- prevents kernel manipulation
- prevents unauthorized automation
- prevents silent data exfiltration
- ensures all actions are user-approved and logged

Residual risks are limited to user decisions and physical access scenarios, both of which are mitigated through permissions, isolation, and TPM-bound auditing.

---

10. Implementation Roadmap

The implementation roadmap defines the sequential phases required to design, build, validate, and deploy the Local Copilot API.
Each phase contains clear engineering tasks, deliverables, and security checkpoints.
No phase can be skipped, and each depends on the successful completion of the previous one.

Phase 1 — Internal Prototype

Objectives

- Establish the foundational architecture.
- Validate feasibility within Windows 11 security constraints.
- Build a minimal, functional execution pipeline.

Engineering Tasks

- Define the complete API surface (allowed operations only).
- Implement the VBS-isolated Local Execution Layer (LEL).
- Implement the Command Validation Layer.
- Implement the Permission Gateway (basic version).
- Implement TPM-bound audit logging.
- Integrate with Windows Security Baseline checks.
- Create internal debugging tools for sandbox inspection.

Security Checkpoints

- Verify VBS isolation boundaries.
- Validate that no arbitrary code execution is possible.
- Confirm that API activation fails when any security module is disabled.

Deliverables

- Working internal prototype.
- Architecture validation report.
- Initial threat model verification.

Phase 2 — Developer Preview

Objectives

- Expose the API to a controlled group of developers.
- Validate API usability, stability, and safety.
- Collect feedback for refinement.

Engineering Tasks

- Publish API documentation (technical only).
- Provide a developer SDK with:
  - type definitions
  - command schemas
  - test harness
  - sandbox simulation tools
- Implement detailed error reporting.
- Expand audit log structure.
- Add granular permission prompts.

Security Checkpoints

- Validate API misuse scenarios.
- Perform internal red-team testing.
- Confirm that all operations remain deterministic.

Deliverables

- Developer Preview build.
- SDK + documentation.
- Security assessment report.

Phase 3 — Windows Integration

Objectives

- Integrate the Local Copilot API into Windows 11 as a system component.
- Provide user-facing configuration and security controls.

Engineering Tasks

- Integrate API activation into Windows Settings.
- Add UI for permission management.
- Add UI for audit log review and export.
- Implement system notifications for security state changes.
- Integrate with Windows Update for servicing.
- Add localization and accessibility support.

Security Checkpoints

- Validate UI permission flows.
- Confirm that API deactivates on security degradation.
- Validate audit log integrity under real-world conditions.

Deliverables

- Windows Insider build with full integration.
- User-facing configuration interface.
- Updated security baseline documentation.

Phase 4 — Enterprise Release

Objectives

- Provide enterprise-grade controls, policies, and deployment tools.
- Ensure compliance with regulatory and organizational requirements.

Engineering Tasks

- Integrate with Group Policy and MDM frameworks.
- Add enterprise permission templates.
- Add enterprise audit log export and retention controls.
- Provide compliance documentation for:
  - EU digital sovereignty
  - public sector requirements
  - regulated industries
- Implement enterprise-grade logging (SIEM integration).

Security Checkpoints

- Validate enterprise policy enforcement.
- Confirm compatibility with hardened security environments.
- Perform external security audits.

Deliverables

- Enterprise-ready release.
- Compliance documentation.
- Final security certification package.

Phase 5 — Long-Term Maintenance

Objectives

- Maintain security, stability, and compatibility.
- Expand the API surface cautiously based on validated use cases.

Engineering Tasks

- Regular security reviews.
- Controlled expansion of allowed operations.
- Continuous VBS and sandbox hardening.
- Telemetry (local only) for error analysis.
- Backward-compatible API versioning.

Security Checkpoints

- Annual threat model updates.
- Continuous red-team testing.
- Regression testing for sandbox boundaries.

Deliverables

- Long-term support plan.
- Versioned API roadmap.
- Updated documentation and security guidelines.

Summary:
The roadmap defines a structured, multi-phase approach that ensures the Local Copilot API is built securely, validated thoroughly, integrated cleanly into Windows, and deployed safely across consumer and enterprise environments.

---

11. Conclusion

The Local Copilot API provides a complete, secure, and deterministic architecture for enabling local automation on Windows 11.
It closes the functional gap between conversational AI and real system interaction while maintaining strict security guarantees enforced by hardware-bound protections and OS-level isolation.

The architecture is based on:

- a predefined, restricted API surface
- a VBS-isolated Local Execution Layer
- a mandatory permission gateway
- TPM-bound audit logging
- strict activation requirements tied to Windows 11 security modules

These components ensure that all operations are:

- predictable
- auditable
- user-controlled
- non-escalating
- compliant with regulatory requirements

The system prevents:

- arbitrary code execution
- privilege escalation
- kernel manipulation
- unauthorized automation
- silent data exfiltration

Residual risks are limited to user-approved destructive actions and physical access scenarios, both mitigated through explicit permissions, isolation, and hardware-anchored integrity.

From a strategic perspective, the architecture leverages existing Windows 11 security infrastructure and positions Microsoft as the first major vendor to provide a secure, OS-integrated AI automation layer that satisfies enterprise and EU sovereignty requirements.

The design is technically feasible, aligned with Windows' security model, and extensible for future controlled expansion of capabilities.

This document defines the complete foundation for implementation, validation, and deployment of the Local Copilot API.

---

12. Appendix A — Extended Technical Notes

This appendix provides additional technical details that support the main architecture.
It includes clarifications, constraints, and engineering considerations that do not fit into the primary sections but are essential for implementation.

12.1 Execution Boundaries

The Local Execution Layer (LEL) must operate under strict boundaries:

- No dynamic memory allocation outside sandbox limits.
- No access to kernel handles.
- No access to raw device interfaces.
- No ability to open network sockets unless explicitly permitted.
- No ability to load or link external libraries.
- No ability to spawn or manage processes.

These boundaries ensure deterministic behavior and prevent privilege escalation.

12.2 Command Serialization Format

All commands must be serialized using a deterministic, schema-validated format.
Recommended structure:
Json

```
{
  "operation": "string",
  "parameters": {
    "...": "..."
  },
  "context": {
    "user_id": "SID",
    "session_id": "GUID",
    "timestamp": "ISO-8601"
  }
}
```

Constraints:

- No embedded scripts.
- No binary payloads.
- No nested operations.
- No recursion.

12.3 Permission Model Details

Permissions must be:

- operation-specific
- path-specific
- time-limited
- revocable

Example permission entry:
Json

```
{
  "operation": "write_file",
  "path": "C:\\Users\\User\\Documents\\Report.txt",
  "expires": "2026-03-13T18:00:00Z",
  "granted_by_user": true
}
```

No wildcard permissions are allowed.

12.4 Audit Log Structure

Audit logs must include:

- operation name
- parameters (sanitized)
- execution result
- timestamp
- user confirmation state
- integrity hash
- sandbox instance ID

Example entry:
Json

```
{
  "timestamp": "2026-03-13T18:22:00Z",
  "operation": "create_folder",
  "parameters": {
    "path": "C:\\Users\\User\\Projects\\NewProject"
  },
  "result": "success",
  "user_confirmed": true,
  "sandbox_id": "VBS-INSTANCE-7F3A",
  "integrity_hash": "TPM-HASH-BASE64"
}
```

Logs must be:

- append-only
- TPM-bound
- exportable only with explicit user consent

12.5 Sandbox Memory Model

The sandbox must use:

- isolated memory pages
- no shared memory with host processes
- no direct pointer passing
- strict boundary checks
- deterministic memory allocation patterns

Memory corruption must trigger:

- sandbox termination
- audit log entry
- API deactivation until restart

12.6 Error Handling Model

Errors must be:

- deterministic
- structured
- non-leaking (no internal state exposure)

Example error:
Json

```
{
  "error": "ACCESS_DENIED",
  "details": "Operation requires user confirmation.",
  "timestamp": "2026-03-13T18:22:00Z"
}
```

No stack traces or internal identifiers may be exposed.

12.7 API Surface Expansion Rules

The API may only be expanded if:

1. The new operation is deterministic.
2. The new operation does not allow code execution.
3. The new operation does not modify system integrity.
4. The new operation can be sandboxed.
5. The new operation can be logged.
6. The new operation can be permission-gated.

If any condition fails → the operation cannot be added.

12.8 Compliance Considerations

The architecture supports:

- EU data sovereignty
- GDPR local processing
- public sector isolation requirements
- enterprise auditability
- zero-trust operational models

No data leaves the device unless explicitly exported by the user.

12.9 Performance Considerations

The sandbox must:

- initialize in under 200 ms
- execute simple operations in under 50 ms
- maintain low memory footprint (< 150 MB)
- avoid blocking UI threads
- support concurrent isolated instances

Performance must never compromise security.

12.10 Summary

Appendix A provides the extended technical constraints and structures required to implement the Local Copilot API securely and predictably.
These details ensure that the system remains deterministic, auditable, and compliant with strict security boundaries.

---

13. Appendix B — API Schema (Technical Draft)

This appendix defines the technical schema for the Local Copilot API.
It specifies the structure, validation rules, and constraints for all commands executed through the Local Execution Layer (LEL).
The schema is deterministic, strictly validated, and designed to prevent arbitrary code execution.

13.1 Schema Overview

All API operations follow a unified schema:

Json

```
{
  "operation": "string",
  "parameters": {
    "...": "..."
  },
  "context": {
    "session_id": "GUID",
    "user_sid": "SID",
    "timestamp": "ISO-8601"
  },
  "metadata": {
    "request_id": "GUID",
    "api_version": "1.0"
  }
}
```

Requirements:

- `operation` must match an approved operation name.
- `parameters` must match the operation-specific schema.
- `context` is generated by Copilot and validated by the system.
- `metadata` ensures traceability and versioning.

No additional fields are allowed.

13.2 Operation Categories

The API surface is divided into categories:

- `file.*`
- `app.*`
- `system.*`
- `workflow.*`

Each category contains deterministic operations with strict parameter validation.

13.3 File Operations Schema

13.3.1 Create File

Json

```
{
  "operation": "file.create",
  "parameters": {
    "path": "string"
  }
}
```

Constraints:

- Path must be absolute.
- Path must be inside a user-approved directory.
- File extension must be non-executable.

13.3.2 Write File

Json

```
{
  "operation": "file.write",
  "parameters": {
    "path": "string",
    "content": "string"
  }
}
```

Constraints:

- Max content size: 1 MB.
- No binary data.
- No script-like patterns (validated via static analysis).

13.3.3 Read File

Json

```
{
  "operation": "file.read",
  "parameters": {
    "path": "string"
  }
}
```

Constraints:

- Read-only.
- No access to system directories.

13.3.4 Delete File

Json

```
{
  "operation": "file.delete",
  "parameters": {
    "path": "string"
  }
}
```

Constraints:

- Requires explicit user confirmation.
- Cannot target protected directories.

13.4 Application Control Schema

13.4.1 Launch Application

Json

```
{
  "operation": "app.launch",
  "parameters": {
    "path": "string",
    "arguments": ["string"]
  }
}
```

Constraints:

- Path must be Smart App Control-approved.
- Arguments must be static strings (no dynamic code).

13.4.2 Close Application

Json

```
{
  "operation": "app.close",
  "parameters": {
    "process_id": "integer"
  }
}
```

Constraints:

- Only user-owned processes.
- No system processes.

13.4.3 Switch Window Focus

Json

```
{
  "operation": "app.focus",
  "parameters": {
    "window_id": "string"
  }
}
```

Constraints:

- Window must belong to the user session.

13.5 System Utilities Schema

13.5.1 Query System Info

Json

```
{
  "operation": "system.info",
  "parameters": {}
}
```

Constraints:

- Read-only.
- No hardware identifiers beyond what is necessary.

13.5.2 Trigger Maintenance Task

Json

```
{
  "operation": "system.maintenance",
  "parameters": {
    "task": "cleanup_temp | cleanup_logs"
  }
}
```

Constraints:

- Only predefined tasks.
- No registry or driver interaction.

13.6 Workflow Automation Schema

13.6.1 Batch Rename

Json

```
{
  "operation": "workflow.batch_rename",
  "parameters": {
    "directory": "string",
    "pattern": "string",
    "replacement": "string"
  }
}
```

Constraints:

- Directory must be user-approved.
- No regex execution; only simple patterns.

13.6.2 Organize Files

Json

```
{
  "operation": "workflow.organize",
  "parameters": {
    "directory": "string",
    "rules": [
      {
        "match": "extension | date | size",
        "value": "string",
        "target": "string"
      }
    ]
  }
}
```

Constraints:

- Rules must be static.
- No dynamic evaluation.

13.7 Validation Rules

Every command must pass:

1. **Schema validation**
2. **Operation whitelist check**
3. **Parameter type validation**
4. **Path safety validation**
5. **Permission gateway approval**
6. **Sandbox compatibility check**
7. **Audit log entry creation**

If any validation fails → the command is rejected.

13.8 Error Schema

All errors follow a unified structure:
Json

```
{
  "error": "string",
  "details": "string",
  "timestamp": "ISO-8601",
  "request_id": "GUID"
}
```

No stack traces or internal identifiers are exposed.

13.9 Summary

Appendix B defines the complete technical schema for the Local Copilot API.
It ensures deterministic behavior, strict validation, and safe execution across all supported operations.

---

14. Appendix C — SDK Structure & Developer Integration

This appendix defines the structure, components, and integration model of the Local Copilot API SDK.
The SDK provides developers with deterministic, sandbox-safe interfaces for interacting with the Local Execution Layer (LEL).
It does not allow arbitrary code execution and exposes only validated, predefined operations.

14.1 SDK Objectives

The SDK must:

- expose a stable, versioned API surface
- enforce schema validation client-side
- provide deterministic behavior
- prevent misuse or unsafe command construction
- integrate with developer tools without exposing internal system details
- support enterprise deployment and policy enforcement

The SDK is not a scripting engine.
It is a structured interface for constructing valid commands.

14.2 SDK Directory Structure

A recommended SDK structure:
Text

```
/sdk
  /core
    api_client.cs
    command_builder.cs
    schema_validator.cs
    error_codes.cs
  /schemas
    file.json
    app.json
    system.json
    workflow.json
  /utils
    path_sanitizer.cs
    sandbox_checker.cs
    permission_request.cs
  /samples
    create_file.cs
    batch_rename.cs
    launch_app.cs
  /docs
    api_reference.md
    integration_guide.md
    security_requirements.md
```

14.3 Core Components

14.3.1 API Client

Responsible for:

- sending commands to the Local Execution Layer
- receiving structured responses
- handling deterministic error codes
- enforcing version compatibility

Example interface:

```
ApiClient.Execute(Command cmd) -> ApiResponse
```

14.3.2 Command Builder

Ensures that all commands:

- match the schema
- contain only allowed fields
- use validated parameter types
- cannot embed scripts or binary payloads

Example:
csharp

```csharp
var cmd = CommandBuilder
    .Operation("file.create")
    .With("path", "C:\\Users\\User\\Documents\\New.txt")
    .Build();
```

Invalid commands must fail at build time.

14.3.3 Schema Validator

Validates commands against JSON schemas stored in `/schemas`.

Validation rules:

- required fields must exist
- no additional fields allowed
- type checks must pass
- path safety rules must pass
- operation must be whitelisted

If validation fails → command is rejected before execution.

14.4 Utility Components

14.4.1 Path Sanitizer

Ensures:

- no traversal (`..`)
- no UNC paths
- no system directories
- no executable extensions
- no hidden device paths (`\\.\`)

14.4.2 Sandbox Checker

Checks:

- VBS state
- API activation state
- security module compliance

If sandbox is inactive → SDK refuses to send commands.

14.4.3 Permission Request Helper

Provides a structured way to request permissions from the user.

Example:
csharp

```csharp
PermissionRequest
    .For("file.write")
    .Path("C:\\Users\\User\\Documents\\Report.txt")
    .Request();
```

The SDK cannot auto-approve permissions.

14.5 Developer Workflow

A typical workflow:

1. Developer constructs a command using `CommandBuilder`.
2. SDK validates the command against schemas.
3. SDK checks sandbox activation.
4. SDK sends the command to the Local Execution Layer.
5. Permission Gateway prompts the user.
6. LEL executes the operation.
7. SDK receives a structured response.
8. SDK logs the result (local only).

14.6 Error Handling

All errors follow deterministic codes:

```
INVALID_SCHEMA
ACCESS_DENIED
SANDBOX_INACTIVE
UNSUPPORTED_OPERATION
PATH_NOT_ALLOWED
PERMISSION_REQUIRED
EXECUTION_FAILED
```

No stack traces or internal identifiers are exposed.

14.7 Versioning Model

The SDK uses semantic versioning:

- MAJOR: breaking changes
- MINOR: new operations added
- PATCH: bug fixes

The Local Execution Layer must reject incompatible SDK versions.

14.8 Enterprise Integration

Enterprises can:

- enforce allowed operations via Group Policy
- restrict paths
- pre-approve certain workflows
- disable specific categories (e.g., `app.*`)
- enforce audit log retention

The SDK must respect all enterprise policies.

14.9 Summary

Appendix C defines the SDK structure and integration model.
The SDK ensures that developers can interact with the Local Copilot API safely, deterministically, and within strict security boundaries.
It prevents misuse, enforces schema compliance, and integrates cleanly with the sandboxed execution environment.

---

15. Appendix D — Permission Model (Deep Technical Specification)

This appendix defines the full technical structure, rules, and constraints of the Permission Model used by the Local Copilot API.
The Permission Gateway is the central enforcement mechanism that ensures all operations are explicitly approved, traceable, and bounded.

15.1 Permission Model Overview

The Permission Model enforces:

- explicit user approval
- operation-specific restrictions
- path-level granularity
- time-limited validity
- revocation at any time
- auditability of all decisions

Permissions are never implicit.
Permissions are never inherited.
Permissions are never global.

15.2 Permission Types

There are four permission types:

1. **Operation Permission**
   Grants access to a specific API operation.
2. **Path Permission**
   Grants access to a specific file or directory.
3. **Application Permission**
   Grants access to launch or close a specific application.
4. **Network Permission** (optional, disabled by default)
   Grants access to predefined network operations.

No wildcard permissions are allowed.

15.3 Permission Structure

A permission entry must follow this schema:
Json

```
{
  "permission_id": "GUID",
  "operation": "string",
  "scope": {
    "path": "string | null",
    "application": "string | null",
    "network_target": "string | null"
  },
  "constraints": {
    "expires": "ISO-8601",
    "max_executions": "integer | null"
  },
  "granted_by_user": true,
  "timestamp": "ISO-8601"
}
```

Requirements:

- `operation` must match an approved API operation.
- `scope` must be explicit and cannot contain wildcards.
- `expires` must be finite (no infinite permissions).
- `max_executions` defaults to 1 if omitted.
- `granted_by_user` must always be `true`.

15.4 Permission Lifecycles

Permissions follow a strict lifecycle:

1. Requested

- Copilot generates a permission request.
- User sees a preview of the intended action.

2. Granted

- User explicitly approves the request.
- Permission is stored in the local permission store.

3. Used

- Permission is consumed when the operation executes.
- `max_executions` decrements.

4. Expired or Exhausted

- Permission becomes invalid when:
  - expiration time is reached
  - execution count reaches zero

5. Revoked

- User can revoke any permission at any time.

Expired or revoked permissions cannot be reactivated.

15.5 Permission Store

Permissions are stored in a secure, local, encrypted store.

Requirements:

- Encryption must use TPM-protected keys.
- Store must be append-only.
- Deletion must be logged.
- No cloud synchronization.
- No export without explicit user consent.

15.6 Permission Evaluation Rules

Before any operation executes, the Permission Gateway evaluates:

1. **Is there a matching permission?**
2. **Is the permission still valid?**
3. **Does the scope match exactly?**
4. **Does the operation exceed allowed parameters?**
5. **Has the user revoked the permission?**
6. **Is the sandbox active?**
7. **Is the system security baseline intact?**

If any check fails → **operation is denied**.

15.7 Permission Request Schema

A permission request must follow this structure:
Json

```
{
  "request_id": "GUID",
  "operation": "string",
  "scope": {
    "path": "string | null",
    "application": "string | null",
    "network_target": "string | null"
  },
  "reason": "string",
  "timestamp": "ISO-8601"
}
```

Requirements:

- `reason` must be human-readable.
- No hidden fields.
- No dynamic code.
- No embedded scripts.

15.8 User Interface Requirements

The permission prompt must:

- show the exact operation
- show the exact path or application
- show the reason
- show the expiration time
- show the number of allowed executions
- require explicit user action (Approve / Deny)
- never auto-approve
- never hide destructive actions

The UI must not:

- group multiple permissions
- pre-select approval
- obscure details
- allow silent approval

15.9 Enterprise Policy Integration

Enterprises may enforce:

- deny-lists for operations
- deny-lists for paths
- deny-lists for applications
- maximum permission durations
- forced logging
- mandatory approval workflows

The Permission Gateway must respect all enterprise policies.

15.10 Revocation Model

Users can revoke permissions:

- individually
- by category
- by application
- by path
- globally (emergency revoke)

Revocation must:

- take effect immediately
- be logged
- invalidate all pending operations

15.11 Security Guarantees

The Permission Model guarantees:

- no operation executes without explicit approval
- no permission is persistent without user consent
- no permission can be broadened by Copilot
- no permission can bypass sandbox boundaries
- all permissions are auditable
- all permissions are revocable

15.12 Summary

Appendix D defines the complete technical specification of the Permission Model.
It ensures that all operations executed through the Local Copilot API are explicitly approved, strictly scoped, time-limited, revocable, and fully auditable.

---

16. Appendix E — Sandbox Architecture (Low-Level Specification)

This appendix defines the low-level architecture of the sandbox environment used by the Local Execution Layer (LEL).
The sandbox is responsible for isolating all operations, preventing privilege escalation, and ensuring deterministic execution.

16.1 Sandbox Objectives

The sandbox must:

- isolate all execution from the Windows kernel
- prevent arbitrary code execution
- enforce strict memory boundaries
- restrict system calls to a predefined whitelist
- ensure deterministic behavior
- provide verifiable integrity
- terminate safely on violation

The sandbox is not a VM and not a container.
It is a **VBS-backed isolated execution environment** with a minimal, fixed capability set.

16.2 Sandbox Architecture Overview

The sandbox consists of:

1. **Isolated Execution Environment (IEE)**
2. **System Call Proxy Layer (SCPL)**
3. **Memory Guard Module (MGM)**
4. **Integrity Enforcement Module (IEM)**
5. **Deterministic Runtime Engine (DRE)**

Each component enforces a different boundary.

16.3 Isolated Execution Environment (IEE)

The IEE is a VBS-protected memory region.

Properties:

- inaccessible to normal processes
- inaccessible to kernel drivers
- no shared memory
- no direct hardware access
- no dynamic linking
- no JIT compilation

Allowed operations:

- fixed, pre-compiled routines
- deterministic memory allocation
- static data tables

Forbidden:

- loading DLLs
- executing EXEs
- spawning threads outside the sandbox
- allocating executable memory

16.4 System Call Proxy Layer (SCPL)

The SCPL is the only interface between the sandbox and Windows.

Responsibilities:

- translate approved operations into safe system calls
- enforce parameter validation
- block all unapproved system calls
- sanitize all inputs and outputs

System Call Rules:

- only whitelisted calls allowed
- no direct registry access
- no raw file handles
- no device I/O
- no network sockets unless explicitly permitted

Example allowed calls:

- CreateDirectoryW
- ReadFile (restricted)
- WriteFile (restricted)
- GetFileAttributesExW
- ShellExecute (restricted, SAC-validated)

16.5 Memory Guard Module (MGM)

The MGM enforces strict memory safety.

Features:

- bounds checking on all buffers
- no pointer arithmetic
- no raw pointer exposure
- fixed-size memory pools
- deterministic allocation patterns

Violation Handling:

- immediate sandbox termination
- audit log entry
- API deactivation until restart

16.6 Integrity Enforcement Module (IEM)

The IEM ensures that:

- sandbox code is unmodified
- sandbox memory is unmodified
- sandbox configuration is unmodified

Mechanisms:

- code integrity checksums
- memory region hashing
- TPM-anchored integrity measurements
- periodic re-validation

If integrity fails → sandbox shuts down.

16.7 Deterministic Runtime Engine (DRE)

The DRE executes operations inside the sandbox.

Requirements:

- no nondeterministic behavior
- no access to system time (except via SCPL)
- no random number generation
- no concurrency outside controlled execution

Execution Model:

- single-threaded
- synchronous
- atomic per operation

This ensures reproducibility and auditability.

16.8 Sandbox Initialization Sequence

1. Validate Windows security baseline.
2. Validate VBS state.
3. Allocate isolated memory region.
4. Load pre-compiled sandbox runtime.
5. Verify integrity hashes.
6. Initialize SCPL.
7. Initialize MGM.
8. Initialize DRE.
9. Register sandbox instance ID.
10. Enter ready state.

If any step fails → sandbox does not start.

16.9 Sandbox Termination Conditions

The sandbox terminates when:

- integrity checks fail
- memory violation occurs
- unauthorized system call is attempted
- permission is revoked mid-operation
- system security baseline degrades
- user disables the Local Copilot API

Termination triggers:

- audit log entry
- invalidation of pending operations
- API deactivation until restart

16.10 Sandbox Performance Constraints

The sandbox must:

- initialize in < 200 ms
- execute simple operations in < 50 ms
- maintain memory footprint < 150 MB
- avoid blocking UI threads
- support multiple isolated instances

Performance must not compromise security.

16.11 Summary

Appendix E defines the low-level sandbox architecture.
The sandbox enforces strict isolation, deterministic execution, and hardware-anchored integrity.
It prevents privilege escalation, arbitrary code execution, and unauthorized system interaction.

---

17. Appendix F — Audit Log Integrity & TPM Binding

This appendix defines the technical mechanisms that ensure the audit log is tamper-resistant, integrity-verified, and cryptographically anchored to the device's Trusted Platform Module (TPM).
The audit log is a core security component of the Local Copilot API.

17.1 Audit Log Objectives

The audit log must:

- record every executed operation
- ensure entries cannot be modified or deleted silently
- provide cryptographic integrity
- support forensic analysis
- remain local unless explicitly exported
- survive system restarts
- detect tampering attempts

The log is not optional.
The API cannot operate without it.

17.2 Log Entry Structure

Each entry follows a strict schema:
Json

```
{
  "entry_id": "GUID",
  "timestamp": "ISO-8601",
  "operation": "string",
  "parameters": { ... },
  "result": "success | failure",
  "user_confirmed": true | false,
  "sandbox_id": "string",
  "integrity_hash": "BASE64",
  "previous_hash": "BASE64"
}
```

Requirements:

- `parameters` must be sanitized (no sensitive data).
- `integrity_hash` must be TPM-sealed.
- `previous_hash` creates a hash chain.

This forms a **cryptographically linked log chain**.

17.3 Hash Chain Mechanism

Each entry includes:

- its own hash
- the hash of the previous entry

This creates a tamper-evident chain:

```
Entry 1 → Entry 2 → Entry 3 → ... → Entry N
```

If any entry is modified:

- all subsequent hashes break
- integrity verification fails
- API deactivates

17.4 TPM Binding

The audit log uses TPM-protected keys.

TPM Functions Used:

- **TPM2_CreatePrimary**
- **TPM2_Create**
- **TPM2_Load**
- **TPM2_Seal / TPM2_Unseal**
- **TPM2_Sign**

TPM Guarantees:

- keys cannot be exported
- keys cannot be duplicated
- keys cannot be used outside the device
- sealing binds keys to system state

This ensures:

- logs cannot be forged
- logs cannot be replayed on another device
- logs cannot be decrypted externally

17.5 Integrity Hash Calculation

The integrity hash is computed as:

```
H = SHA-256(
    entry_id +
    timestamp +
    operation +
    sanitized_parameters +
    result +
    user_confirmed +
    sandbox_id +
    previous_hash
)
```

Then sealed with TPM:

```
TPM_Seal(H)
```

The sealed hash is stored in the log entry.

17.6 Log Storage Requirements

The log must be stored:

- locally
- encrypted at rest
- append-only
- in a protected directory
- with ACLs restricting access to SYSTEM only

No cloud sync.
No remote storage.
No silent deletion.

17.7 Log Rotation

When the log reaches a size threshold:

- a new log file is created
- the previous log is sealed
- a TPM-signed summary is generated

Rotation must not break the hash chain.

17.8 Export Mechanism

Export requires:

- explicit user approval
- TPM unsealing
- integrity verification
- export in read-only format

Enterprises may enforce:

- export frequency
- export retention
- export encryption

17.9 Tamper Detection

Tampering is detected when:

- hash chain breaks
- TPM unseal fails
- log file is modified externally
- log file is truncated
- log file is replaced

On detection:

1. API deactivates
2. sandbox terminates
3. event is logged in Windows Event Log
4. user is notified

17.10 Forensic Guarantees

The audit log provides:

- chronological traceability
- cryptographic integrity
- device-bound authenticity
- non-repudiation of actions
- verifiable sandbox behavior

This enables:

- internal audits
- compliance verification
- incident response
- forensic reconstruction

17.11 Summary

Appendix F defines the complete integrity and TPM-binding model for the audit log.
The log is cryptographically chained, hardware-anchored, tamper-evident, and mandatory for all Local Copilot API operations.
Any tampering attempt results in immediate API deactivation and full forensic traceability.

---

18. Appendix G — Security Baseline Enforcement Logic

This appendix defines the exact logic used to verify, enforce, and continuously monitor the Windows 11 security baseline required for the Local Copilot API.
The API must deactivate immediately if any required security component becomes unavailable, misconfigured, or compromised.

18.1 Security Baseline Components

The baseline consists of the following mandatory components:

- TPM 2.0
- Secure Boot
- Virtualization-Based Security (VBS)
- Memory Integrity (HVCI)
- Core Isolation
- Smart App Control (SAC)
- No unsigned kernel drivers
- No integrity violations in Windows Security Baseline

All components must be active and verified.

18.2 Enforcement Model Overview

The enforcement logic operates at three levels:

1. **Startup Validation**
2. **Continuous Monitoring**
3. **Event-Driven Revalidation**

If any level fails → API deactivates.

18.3 Startup Validation Logic

At system startup or API initialization, the following sequence is executed:

```
if !TPM_Ready() → FAIL
if !SecureBoot_Enabled() → FAIL
if !VBS_Active() → FAIL
if !HVCI_Enabled() → FAIL
if !CoreIsolation_Enabled() → FAIL
if !SmartAppControl_ActiveOrEval() → FAIL
if UnsignedKernelDriversPresent() → FAIL
if SecurityBaselineIntegrityBroken() → FAIL

→ PASS (API may activate)
```

Deactivation Procedure:

1. Terminate sandbox
2. Invalidate pending operations
3. Write audit log entry
4. Notify user
5. Require system restart to reactivate

18.5 Event-Driven Revalidation

Certain system events trigger immediate revalidation:

- driver installation
- firmware update
- Windows Update
- SAC mode change
- VBS configuration change
- registry modifications to security components
- kernel memory protection changes

Event handler:

```
on_security_event:
    if !ValidateBaseline() → DEACTIVATE
```

18.6 Integrity Measurement Logic

Integrity is validated using:

- TPM PCR values
- Secure Boot policy
- VBS configuration hashes
- HVCI enforcement state
- kernel driver signature table
- Windows Security Baseline manifest

Validation function:

```
ValidateBaseline():
    return (
        TPM_PCRs_MatchExpected() &&
        SecureBoot_PolicyValid() &&
        VBS_ConfigHashValid() &&
        HVCI_Enforced() &&
        KernelDrivers_AllSigned() &&
        BaselineManifest_Intact()
    )
```

If any check fails → baseline invalid.

18.7 Smart App Control Enforcement

SAC must be in:

- **On**
- or **Evaluation** mode

If SAC enters **Off**:

- API deactivates
- sandbox terminates
- audit entry is written

SAC ensures:

- only trusted binaries can execute
- no unverified EXEs or DLLs can be launched
- no dynamic code injection

18.8 Unsigned Driver Detection Logic

Unsigned drivers are detected via:

- kernel driver enumeration
- signature verification
- cross-checking with Windows driver trust store

If any unsigned driver is found:

```
DEACTIVATE_API()
```

Unsigned drivers represent a direct kernel-level threat.

18.9 Baseline Degradation Handling

If the baseline degrades during runtime:

- API deactivates immediately
- no grace period
- no partial functionality
- no fallback mode

This prevents:

- privilege escalation
- kernel tampering
- sandbox escape attempts

18.10 Recovery Logic

Recovery requires:

1. Restoring all baseline components
2. Restarting the system
3. Passing full startup validation

No automatic recovery is allowed.

18.11 Summary

Appendix G defines the complete enforcement logic for the Windows 11 security baseline.
The Local Copilot API activates only when all security components are verified and remains active only as long as the baseline stays intact.
Any deviation results in immediate deactivation, sandbox termination, and full audit logging.

---

19. Appendix H — Failure Modes & Recovery Logic

This appendix defines all failure modes of the Local Copilot API, the sandbox, the permission system, and the security baseline.
It also defines the exact recovery logic for each failure type.
No automatic or silent recovery is allowed.

19.1 Failure Mode Categories

Failures are grouped into six categories:

1. **Security Baseline Failures**
2. **Sandbox Integrity Failures**
3. **Permission Model Failures**
4. **System Call Violations**
5. **Audit Log Failures**
6. **Internal Runtime Failures**

Each category has strict handling rules.

19.2 Security Baseline Failures

Triggered when:

- TPM becomes unavailable
- Secure Boot is disabled
- VBS is disabled
- HVCI is disabled
- Core Isolation is disabled
- Smart App Control switches to Off
- unsigned kernel drivers appear
- baseline integrity hash changes

Response:

1. Immediately deactivate API
2. Terminate sandbox
3. Invalidate pending operations
4. Write audit log entry
5. Notify user
6. Require system restart

Recovery:

- Restore baseline
- Restart system
- Pass full startup validation

No partial recovery.

19.3 Sandbox Integrity Failures

Triggered when:

- memory violation occurs
- integrity hash mismatch
- unauthorized system call attempted
- sandbox configuration modified
- sandbox code tampered
- sandbox runtime corrupted

Response:

1. Terminate sandbox
2. Deactivate API
3. Log failure with TPM-bound entry
4. Notify user

Recovery:

- Restart system
- Reinitialize sandbox
- Pass integrity validation

No auto-restart of sandbox.

19.4 Permission Model Failures

Triggered when:

- permission expired
- permission revoked
- permission scope mismatch
- permission store corrupted
- permission request malformed
- permission tampering detected

Response:

- Deny operation
- Log event
- Notify user (if interactive)

Recovery:

- User must re-approve permission
- Permission store must pass integrity check

No automatic re-granting.

19.5 System Call Violations

Triggered when:

- operation attempts a non-whitelisted system call
- parameters violate safety rules
- path is outside allowed scope
- application is not SAC-approved
- operation attempts to spawn a process
- operation attempts to load a DLL

Response:

1. Abort operation
2. Terminate sandbox instance
3. Log violation
4. Deactivate API (depending on severity)

Recovery:

- Restart system
- Reinitialize sandbox

No retry of the violating operation.

19.6 Audit Log Failures

Triggered when:

- hash chain breaks
- TPM unseal fails
- log file modified externally
- log file truncated
- log file replaced
- log rotation integrity fails

Response:

1. Deactivate API
2. Terminate sandbox
3. Notify user
4. Write Windows Event Log entry

Recovery:

- Create new log file
- Seal previous log as corrupted
- Require user approval to continue

No silent repair.

19.7 Internal Runtime Failures

Triggered when:

- deterministic runtime engine (DRE) encounters unexpected state
- memory pool exhaustion
- internal deadlock (should not occur due to single-threaded model)
- corrupted command structure
- schema mismatch
- unexpected null values

Response:

- Abort operation
- Log failure
- Reset sandbox instance

Recovery:

- Reinitialize sandbox
- Validate schemas
- Continue only after successful re-validation

19.8 Failure Severity Levels

Failures are classified:

Level	Description	Action
Critical	Security baseline or integrity failure	Full deactivation
High	Sandbox violation or audit log failure	Sandbox termination + deactivation
Medium	Permission or system call violation	Operation denied
Low	Runtime error	Sandbox reset

No failure is ignored.

19.9 Global Deactivation Conditions

The API must deactivate globally when:

- any critical or high-severity failure occurs
- audit log integrity cannot be verified
- sandbox integrity cannot be verified
- security baseline is degraded
- permission store is corrupted

Deactivation is immediate and non-negotiable.

19.10 Recovery Principles

Recovery must follow these principles:

- **explicit** (never silent)
- **user-controlled**
- **security-first**
- **no automatic reactivation**
- **full revalidation required**

Recovery always requires:

1. Restoring all required security components
2. Restarting the system
3. Passing full startup validation
4. Reinitializing sandbox
5. Revalidating audit log
6. Revalidating permission store

19.11 Summary

Appendix H defines all failure modes and the strict recovery logic.
The system never attempts silent recovery, never ignores violations, and never continues operation under degraded security.
Every failure is logged, traceable, and requires explicit user-controlled recovery.

---

20. Appendix I — Deterministic Execution Guarantees

This appendix defines the mechanisms that ensure all operations executed through the Local Copilot API behave deterministically.
Determinism is essential for auditability, reproducibility, security validation, and enterprise compliance.

The system must guarantee that the same command, under the same conditions, always produces the same result.

20.1 Determinism Objectives

The execution model must ensure:

- no nondeterministic behavior
- no race conditions
- no concurrency outside controlled execution
- no access to system time except via SCPL
- no randomness
- no external dependencies
- no environment-dependent behavior

Determinism is enforced at every layer.

20.2 Deterministic Runtime Engine (DRE)

The DRE enforces deterministic execution through:

- single-threaded execution
- atomic operations
- fixed execution order
- no asynchronous tasks
- no background threads
- no shared memory
- no external state access

Execution Model:

```
execute(operation):
    validate()
    lock()
    run()
    unlock()
    return result
```

No parallelism is allowed inside the sandbox.

20.3 Deterministic Memory Model

The memory model enforces:

- fixed allocation patterns
- static memory pools
- no dynamic resizing
- no pointer arithmetic
- no nondeterministic garbage collection

Memory allocation must follow a predictable sequence.

Example:

```
Pool A → Pool B → Pool C → repeat
```

If allocation fails → sandbox terminates.

20.4 Deterministic System Call Proxy Layer (SCPL)

The SCPL ensures:

- only whitelisted system calls
- fixed parameter validation rules
- fixed return value normalization
- no system-dependent variations

Example:

- File timestamps are normalized to UTC
- Error codes are mapped to deterministic API error codes
- Path resolution is canonicalized

This prevents OS-level nondeterminism from leaking into the sandbox.

20.5 Deterministic Permission Evaluation

Permission evaluation must be:

- atomic
- stateless (beyond permission store)
- order-independent
- reproducible

The same permission request must always produce the same decision.

20.6 Deterministic Audit Logging

Audit logging must:

- use monotonic sequence numbers
- use hash chaining
- use TPM-sealed integrity hashes
- never reorder entries
- never merge entries

The log must reflect the exact execution order.

20.7 Deterministic Error Handling

Errors must:

- follow a fixed schema
- use predefined error codes
- never expose internal state
- never include stack traces
- never include memory addresses

Example:

Json

```
{
  "error": "PATH_NOT_ALLOWED",
  "details": "The specified path is outside the approved scope.",
  "timestamp": "2026-03-13T19:39:00Z",
  "request_id": "GUID"
}
```

No variability is allowed.

20.8 Deterministic File System Behavior

To ensure determinism:

- directory listings must be sorted
- file metadata must be normalized
- symbolic links must be resolved deterministically
- hidden/system files must be filtered consistently

The sandbox must not rely on OS-specific ordering.

20.9 Deterministic Application Control

Application operations must:

- use canonical paths
- validate SAC trust state deterministically
- normalize process identifiers
- reject ambiguous window handles

No OS-dependent heuristics are allowed.

20.10 Deterministic Workflow Automation

Workflow operations must:

- use fixed iteration order
- use stable sorting rules
- avoid parallel processing
- avoid nondeterministic file system queries

Example:

- Batch rename processes files in lexicographical order
- Organize operations apply rules in fixed sequence

20.11 Deterministic Sandbox Initialization

Initialization must follow a fixed sequence:

1. Validate baseline
2. Allocate memory pools
3. Load runtime
4. Verify integrity
5. Initialize SCPL
6. Initialize MGM
7. Initialize DRE
8. Register sandbox ID

Any deviation → initialization fails.

20.12 Deterministic Termination

Termination must:

- follow a fixed shutdown sequence
- flush logs in deterministic order
- release memory pools predictably
- invalidate sandbox ID

No partial shutdown is allowed.

20.13 Deterministic Behavior Under Failure

Failures must:

- produce deterministic error codes
- follow fixed recovery rules
- never produce ambiguous states
- never retry automatically

Example:

- Memory violation → sandbox termination
- Permission mismatch → operation denied

20.14 Summary

Appendix I defines the deterministic execution guarantees required for the Local Copilot API. Determinism ensures:

- reproducibility
- auditability
- security validation
- compliance
- predictable behavior

Every layer of the system enforces deterministic rules, ensuring that operations behave identically across executions, devices, and environments.

---

21. Appendix J — System Call Whitelist (Complete Technical Listing)

This appendix defines the complete, explicit whitelist of system calls that the Local Execution Layer (LEL) may invoke through the System Call Proxy Layer (SCPL).
No system call outside this list may ever be executed.
Any attempt to invoke a non-whitelisted call results in immediate sandbox termination.

The whitelist is intentionally minimal and restricted to deterministic, non-privileged operations.

21.1 Principles of System Call Whitelisting

A system call is allowed only if it satisfies all of the following:

1. **Deterministic**
2. **Non-privileged**
3. **No kernel manipulation**
4. **No device I/O**
5. **No registry modification**
6. **No process creation**
7. **No thread creation**
8. **No dynamic code loading**
9. **No network socket creation**
10. **No access to raw handles**

If any condition fails → the call is forbidden.

21.2 File System Call Whitelist

Allowed calls:

- `CreateDirectoryW`
- `RemoveDirectoryW`
- `GetFileAttributesW`
- `GetFileAttributesExW`
- `SetFileAttributesW`
- `CreateFileW` (restricted: read/write only, no execute, no device paths)
- `ReadFile` (restricted: max buffer size enforced)
- `WriteFile` (restricted: text-only, no binary payloads)
- `CloseHandle`
- `FindFirstFileW`
- `FindNextFileW`
- `FindClose`
- `GetFileInformationByHandle`
- `GetFileSizeEx`
- `SetEndOfFile`
- `FlushFileBuffers`

Forbidden:

- `DeviceIoControl`
- `CreateFileA`
- `NtCreateFile`
- `NtQueryDirectoryFile`
- `NtSetInformationFile`
- any raw NT kernel calls

21.3 Directory Enumeration Whitelist

Allowed:

- `FindFirstFileW`
- `FindNextFileW`
- `FindClose`

Restrictions:

- results must be sorted deterministically
- symbolic links must be resolved safely
- hidden/system files filtered unless explicitly allowed

21.4 Application Control Whitelist

Allowed:

- `ShellExecuteExW` (restricted: SAC-approved binaries only)
- `EnumWindows`
- `GetWindowThreadProcessId`
- `SetForegroundWindow`
- `IsWindow`
- `GetWindowTextW`
- `GetClassNameW`

Forbidden:

- `CreateProcessW`
- `CreateProcessAsUserW`
- `NtCreateUserProcess`
- `LoadLibraryW`
- `LoadLibraryExW`
- `GetProcAddress`
- `SetWindowsHookEx`

No process creation.
No DLL loading.
No code injection.

21.5 System Information Whitelist

Allowed:

- `GetSystemInfo`
- `GlobalMemoryStatusEx`
- `GetDiskFreeSpaceExW`
- `GetVersionExW` (normalized output)
- `QueryPerformanceCounter` (restricted, normalized)
- `QueryPerformanceFrequency`

Forbidden:

- `NtQuerySystemInformation`
- `NtQueryInformationProcess`
- `NtQueryInformationThread`

21.6 Time & Clock Whitelist

Allowed:

- `GetSystemTimePreciseAsFileTime`
- `GetSystemTimeAsFileTime`

Restrictions:

- timestamps normalized to UTC
- no access to timezone configuration
- no access to high-resolution timers beyond deterministic range

Forbidden:

- `GetTickCount64` (nondeterministic drift)
- `timeGetTime`
- `NtQueryTimerResolution`

21.7 Memory Management Whitelist

Allowed:

- `VirtualAlloc` (restricted: non-executable, fixed size, sandbox region only)
- `VirtualFree` (restricted)
- `RtlZeroMemory`
- `RtlCopyMemory`

Forbidden:

- `VirtualProtect`
- `VirtualAllocEx`
- `MapViewOfFile`
- `CreateFileMapping`
- `NtAllocateVirtualMemory`

No executable memory.
No shared memory.
No cross-process memory.

21.8 Registry Access Whitelist

Allowed:

- `RegGetValueW` (read-only, restricted keys)

Forbidden:

- `RegSetValueW`
- `RegCreateKeyW`
- `RegDeleteKeyW`
- `RegOpenKeyExW` (write access)
- any NT registry calls

The sandbox cannot modify the registry.

21.9 Network Call Whitelist

Default: **No network calls allowed.**

If explicitly enabled by user permission:

Allowed:

- `WinHttpOpen`
- `WinHttpConnect`
- `WinHttpOpenRequest`
- `WinHttpSendRequest`
- `WinHttpReceiveResponse`
- `WinHttpReadData`
- `WinHttpCloseHandle`

Restrictions:

- only HTTPS
- only user-approved domains
- no raw sockets
- no DNS manipulation
- no ICMP
- no UDP

Forbidden:

- `socket`
- `connect`
- `send`
- `recv`
- `WSAStartup`
- any Winsock calls

21.10 Forbidden System Call Categories

The following categories are globally forbidden:

- process creation
- thread creation
- DLL loading
- code injection
- raw NT kernel calls
- device I/O
- driver loading
- registry modification
- network socket creation
- WMI access
- COM object creation
- PowerShell hosting
- script execution
- JIT compilation
- GPU compute APIs

Any attempt triggers sandbox termination.

21.11 Enforcement Logic

Before a system call is executed:

```
if call not in whitelist → TERMINATE_SANDBOX
if parameters violate constraints → TERMINATE_SANDBOX
if call attempts privileged operation → TERMINATE_SANDBOX
if call interacts with kernel objects → TERMINATE_SANDBOX
```

No exceptions.

21.12 Summary

Appendix J defines the complete system call whitelist.
Only these calls may be executed by the Local Execution Layer.
Any deviation results in immediate sandbox termination and full audit logging.
This whitelist is intentionally minimal to guarantee security, determinism, and isolation.

22. Appendix K — Path Safety Rules (Canonicalization & Enforcement)

This appendix defines the complete path safety model used by the Local Copilot API.
Path safety rules ensure that file system operations are deterministic, secure, and restricted to explicitly approved locations.
No operation may proceed unless the path passes all validation stages.

22.1 Path Safety Objectives

The path safety model must:

- prevent directory traversal
- prevent access to system directories
- prevent access to device paths
- prevent access to UNC paths
- enforce canonicalization
- enforce deterministic path resolution
- restrict operations to user-approved scopes
- prevent symbolic link abuse
- prevent NT path bypasses

No exceptions.

22.2 Path Canonicalization Pipeline

Every path must pass through a strict canonicalization pipeline:

```
input_path
    ↓
normalize_slashes()
    ↓
expand_environment_variables()
    ↓
resolve_relative_segments()
    ↓
resolve_symlinks()
    ↓
convert_to_absolute()
    ↓
validate_against_rules()
    ↓
final_canonical_path
```

If any stage fails → operation denied.

22.3 Forbidden Path Types

The following path types are globally forbidden:

1. UNC Paths

```
\\server\share\...
```

2. Device Paths

```
\\.\PhysicalDrive0
\\?\Volume{GUID}
```

3. NT Native Paths

```
\??\C:\Windows
\Device\HarddiskVolume1
```

4. Named Pipes

```
\\.\pipe\...
```

5. Kernel Object Paths

```
\Device\NamedPipe\...
```

6. Registry Pseudo-Paths

```
HKLM:\...
HKCU:\...
```

Any attempt to use these → immediate rejection.

22.4 Forbidden Directories

The following directories are always forbidden:

- `C:\Windows\*`
- `C:\Windows\System32\*`
- `C:\Windows\SysWOW64\*`
- `C:\Program Files\*`
- `C:\Program Files (x86)\*`
- `C:\ProgramData\*`
- `C:\Users\*\AppData\Local\*`
- `C:\Users\*\AppData\Roaming\*`
- `C:\Users\*\AppData\LocalLow\*`
- any hidden/system directory

The sandbox cannot interact with system-critical paths.

22.5 Allowed Base Directories

By default, only the following directories are allowed:

- `C:\Users\<User>\Documents\*`
- `C:\Users\<User>\Desktop\*`
- `C:\Users\<User>\Downloads\*`
- `C:\Users\<User>\Pictures\*`
- `C:\Users\<User>\Videos\*`
- `C:\Users\<User>\Music\*`
- explicitly user-approved directories

No other directories may be accessed.

22.6 Directory Traversal Protection

The following patterns are forbidden:

- `..\`
- `../`
- `.\`
- `./`
- encoded traversal (`%2e%2e\`)
- Unicode homoglyph traversal

Traversal detection is performed **before** canonicalization.

22.7 Symbolic Link Resolution Rules

Symbolic links must be resolved:

- fully
- deterministically
- before permission evaluation

If a symlink resolves to:

- a forbidden directory
- a device path
- a UNC path
- a system directory

→ operation denied.

Symlink chains longer than 5 levels → denied.

22.8 Executable Extension Restrictions

The following extensions are forbidden for write operations:

- `.exe`
- `.dll`
- `.sys`
- `.msi`
- `.bat`
- `.cmd`
- `.ps1`
- `.vbs`
- `.js`
- `.wsf`
- `.scr`

The sandbox cannot create or modify executable files.

22.9 Path Length Restrictions

Maximum allowed canonical path length:

```
260 characters
```

Paths exceeding this limit → denied.

22.10 Canonicalization Rules (Detailed)

1. Normalize slashes

Convert `/` to `\`.

2. Expand environment variables

Only user-level variables allowed.

3. Resolve relative segments

- `.` removed
- `..` forbidden

4. Resolve symlinks

Must resolve to a real, physical path.

5. Convert to absolute

Relative paths forbidden.

6. Validate final path

Must match allowed base directories.

22.11 Path Safety Enforcement Logic

```
canonical = canonicalize(path)
if canonical == INVALID → DENY
if canonical not in allowed_base_dirs → DENY
if canonical matches forbidden_patterns → DENY
if canonical resolves to forbidden_dir → DENY
if extension forbidden → DENY
if permission not granted → DENY
→ ALLOW
```

No fallback.
No heuristics.
No silent correction.

22.12 Example: Valid Path

Input:

```
C:/Users/User/Documents/Project/../Report.txt
```

Canonicalization:

```
C:\Users\User\Documents\Report.txt
```

Allowed.

22.13 Example: Invalid Path (Traversal)

Input:

```
C:\Users\User\Documents\..\..\Windows\System32
```

Rejected at traversal stage.

22.14 Example: Invalid Path (Symlink Escape)

Input:

```
C:\Users\User\Documents\LinkToSystem32\file.txt
```

Symlink resolves to:

```
C:\Windows\System32\file.txt
```

Rejected.

22.15 Summary

Appendix K defines the complete path safety model.
All paths must be canonicalized, validated, and restricted to user-approved directories.
No traversal, no symlink escape, no system directory access, no device paths, and no executable file writes are allowed.
Any violation results in immediate rejection of the operation.

---

23. Appendix L — Application Trust Model (Smart App Control Integration)

This appendix defines how the Local Copilot API integrates with Smart App Control (SAC) to enforce application trust.
The Application Trust Model ensures that only verified, trusted, and policy-compliant applications can be launched or controlled through the Local Execution Layer (LEL).

23.1 Objectives of the Application Trust Model

The model must:

- prevent execution of untrusted binaries
- prevent execution of unsigned or tampered applications
- enforce SAC trust decisions
- ensure deterministic application behavior
- prevent privilege escalation through application launch
- restrict application control to user-approved targets

No exceptions.

23.2 SAC Enforcement Modes

Smart App Control has three modes:

1. **On**
2. **Evaluation**
3. **Off** (forbidden)

The Local Copilot API requires:

- SAC = **On** or **Evaluation**
- SAC = **Off** → API deactivation

23.3 Application Trust Requirements

An application may only be launched or controlled if:

1. It is SAC-approved
2. It is signed with a trusted certificate
3. It has not been tampered with
4. It resides in a non-forbidden directory
5. It matches a user-approved permission entry
6. It is not a script host or interpreter
7. It does not require elevated privileges

If any requirement fails → operation denied.

23.4 Forbidden Application Types

The following application types are globally forbidden:

- PowerShell (`powershell.exe` , `pwsh.exe`)
- Command Prompt (`cmd.exe`)
- Windows Terminal (`wt.exe`)
- WSL (`wsl.exe`)
- Script hosts (`wscript.exe` , `cscript.exe`)
- Scripting engines (`python.exe` , `node.exe` , `ruby.exe` , etc.)
- Installer frameworks (`msiexec.exe`)
- Driver installers
- Debuggers
- Code injection tools
- Any unsigned executable

The sandbox cannot interact with these applications.

23.5 Allowed Application Types

Allowed applications include:

- SAC-approved productivity apps
- SAC-approved UWP apps
- SAC-approved Win32 apps
- SAC-approved system utilities (non-privileged)
- SAC-approved user applications in user directories

Examples:

- Notepad
- Paint
- Calculator
- Office applications
- Browsers (restricted control)
- File Explorer (restricted control)

23.6 Application Launch Validation Pipeline

Before launching an application:

```
1. Canonicalize path
2. Validate path safety
3. Validate SAC trust state
4. Validate digital signature
5. Validate file integrity
6. Validate permission entry
7. Validate non-privileged execution
8. Validate sandbox compatibility
```

If any step fails → deny.

23.7 Application Control Restrictions

Allowed operations:

- launch application
- close application
- switch window focus
- retrieve window metadata

Forbidden operations:

- sending keystrokes
- sending mouse events
- injecting input
- modifying window styles
- enumerating processes outside user session
- interacting with elevated processes

No automation of privileged UI.

23.8 Digital Signature Requirements

The executable must be:

- signed
- timestamped
- verified against Windows trust store
- not revoked
- not expired

Unsigned or self-signed binaries → denied.

23.9 Integrity Verification

Integrity is verified using:

- Authenticode signature
- SHA-256 file hash
- SAC trust decision
- Windows Defender reputation data

If any mismatch occurs → deny.

23.10 Application Permission Model

A permission entry for application launch:
Json

```
{
  "operation": "app.launch",
  "application": "C:\\Program Files\\App\\app.exe",
  "expires": "2026-03-14T12:00:00Z",
  "max_executions": 3,
  "granted_by_user": true
}
```

Rules:

- path must be canonical
- application must be SAC-approved
- permission must be explicit

No wildcard application permissions.

23.11 Window Control Safety Rules

Window control operations must:

- validate window handle
- validate process ownership
- validate session ownership
- reject elevated windows
- reject system windows

Example allowed:

- focusing a Notepad window

Example forbidden:

- focusing Task Manager (elevated)

23.12 SAC State Change Handling

If SAC transitions to **Off**:

1. API deactivates
2. sandbox terminates
3. audit log entry written
4. user notified

Reactivation requires:

- SAC restored to On/Evaluation
- system restart
- full baseline validation

23.13 Summary

Appendix L defines the Application Trust Model.
All application interactions must pass SAC trust checks, digital signature validation, integrity verification, and permission evaluation.
Untrusted, unsigned, privileged, or script-capable applications are strictly forbidden.
Any violation results in immediate denial or sandbox termination.

---

24. Appendix M — Network Operation Restrictions & Allowed Domains Model

This appendix defines the complete network security model for the Local Copilot API.
By default, **all network access is disabled**.
Network operations are only possible when explicitly enabled by the user and must follow strict domain, protocol, and permission constraints.

24.1 Network Model Objectives

The network model must:

- prevent unauthorized data exfiltration
- prevent arbitrary network access
- restrict communication to user-approved domains
- enforce HTTPS-only communication
- enforce deterministic request behavior
- prevent socket-level operations
- prevent DNS manipulation
- ensure full auditability

No fallback to insecure modes.

24.2 Default Network Policy

Default state:

```
NETWORK_ACCESS = DISABLED
```

The Local Copilot API cannot:

- open sockets
- resolve DNS
- send HTTP requests
- receive HTTP responses
- use any network API

All network operations require explicit user permission.

24.3 Network Permission Requirements

A network permission entry must specify:
Json

```
{
  "operation": "network.request",
  "domain": "https://example.com",
  "path": "/api/v1/data",
  "method": "GET | POST",
  "expires": "ISO-8601",
  "max_requests": "integer",
  "granted_by_user": true
}
```

Requirements:

- domain must be HTTPS
- domain must be fully qualified
- no wildcards (`*.example.com` forbidden)
- path must be static
- method must be GET or POST
- no dynamic query parameters

24.4 Allowed Network APIs

If network access is enabled, only the following WinHTTP APIs are allowed:

- `WinHttpOpen`
- `WinHttpConnect`
- `WinHttpOpenRequest`
- `WinHttpSendRequest`
- `WinHttpReceiveResponse`
- `WinHttpReadData`
- `WinHttpCloseHandle`

Restrictions:

- HTTPS only
- TLS 1.2+
- no redirects
- no cookies
- no persistent connections
- no streaming
- no chunked encoding

24.5 Forbidden Network APIs

The following APIs are always forbidden:

- Winsock (`socket` , `connect` , `send` , `recv`)
- raw TCP/UDP
- ICMP
- DNS APIs (`DnsQuery` , `GetAddrInfoW`)
- WebSockets
- HTTP/3
- SMB
- FTP
- SSH
- any COM-based network API
- WMI network operations

Any attempt triggers sandbox termination.

24.6 Domain Validation Rules

Domains must:

- be explicitly approved
- use HTTPS
- resolve to a stable IP (checked once by OS, not by sandbox)
- not be local network addresses
- not be private IP ranges
- not be loopback addresses

Forbidden:

- `http://`
- `localhost`
- `127.0.0.1`
- `10.x.x.x`
- `172.16.x.x – 172.31.x.x`
- `192.168.x.x`
- `.local` domains
- IP-literal URLs

24.7 Request Construction Rules

Requests must:

- use static URLs
- use static headers
- use static payloads
- be fully defined in the permission entry
- not include dynamic content
- not include user identifiers
- not include system identifiers

Example allowed request:

```
GET https://api.example.com/v1/status
```

Example forbidden request:

```
POST `https://example.com/upload?user=123&token=abc` (example.com in Bing)
```

Dynamic parameters → denied.

24.8 Response Handling Rules

Responses must:

- be limited to 1 MB
- be text-only
- be UTF-8
- be sanitized
- be logged
- not trigger follow-up requests
- not contain executable content

Forbidden response types:

- binary
- compressed
- encoded
- scripts
- HTML with active content

24.9 Network Operation Flow

Execution flow:

```
1. Validate permission
2. Validate domain
3. Validate path
4. Validate method
5. Validate HTTPS
6. Construct request
7. Send request via WinHTTP
8. Receive response
9. Sanitize response
10. Log operation
11. Return result
```

Any failure → deny.

24.10 Network Safety Enforcement Logic

Before executing a network request:

```
if NETWORK_ACCESS_DISABLED → DENY
if domain not approved → DENY
if method not allowed → DENY
if path not static → DENY
if protocol != HTTPS → DENY
if IP is private/local → DENY
if payload > 1MB → DENY
if response > 1MB → DENY
if content-type not text → DENY
```

No exceptions.

24.11 Example: Valid Network Request

Permission:

```
domain: "https://api.weather.com"
path: "/v1/current"
method: "GET"
```

Request:

```
GET https://api.weather.com/v1/current
```

Allowed.

24.12 Example: Invalid Network Request (Wildcard Domain)

```
https://*.example.com
```

Denied.

24.13 Example: Invalid Network Request (Local Network)

```
http://192.168.1.10/api` (192.168.1.10 in Bing)
```

Denied.

24.14 Summary

Appendix M defines the complete network restrictions and allowed domain model.
Network access is disabled by default and only enabled through explicit, granular permissions.
All requests must be deterministic, static, HTTPS-only, domain-restricted, and fully auditable.
Any deviation results in immediate denial or sandbox termination.

---

25. Appendix N — Local-Only Telemetry & Error Diagnostics Model

This appendix defines the telemetry and diagnostics model for the Local Copilot API.
All telemetry is **strictly local**, **never transmitted**, **never synced**, and **never exported** unless the user explicitly approves it.
The model exists solely to support debugging, error analysis, and deterministic behavior validation.

25.1 Telemetry Objectives

The telemetry system must:

- operate entirely offline
- store all data locally
- avoid collecting personal data
- avoid collecting content data
- avoid collecting identifiers
- avoid collecting environment fingerprints
- provide deterministic diagnostic information
- support reproducible debugging
- integrate with audit logs without duplication

No cloud, no sync, no remote endpoints.

25.2 Telemetry Categories

Telemetry is divided into four categories:

1. **Execution Metrics**
2. **Error Diagnostics**
3. **Sandbox Health Metrics**
4. **Baseline State Metrics**

Each category is strictly bounded.

25.3 Execution Metrics (Local-Only)

Collected:

- operation name
- execution duration
- sandbox instance ID
- memory usage (sandbox only)
- system call count
- permission evaluation time

Not collected:

- file contents
- user data
- application data
- network data
- identifiers
- environment fingerprints

Execution metrics are stored in a rotating local buffer.

25.4 Error Diagnostics

Diagnostics include:

- error code
- error category
- failing subsystem
- timestamp
- sandbox state snapshot (non-sensitive)
- last successful operation ID

Diagnostics must **never** include:

- stack traces
- memory dumps
- file contents
- user identifiers
- system identifiers
- registry values
- kernel state

Diagnostics are sanitized before storage.

25.5 Sandbox Health Metrics

Collected:

- memory pool usage
- allocation failures
- integrity check results
- system call violation attempts
- sandbox restart count
- sandbox termination reason

Not collected:

- memory contents
- pointer values
- raw system call parameters

25.6 Baseline State Metrics

Collected:

- TPM availability
- Secure Boot state
- VBS state
- HVCI state
- SAC state
- unsigned driver detection flag

Not collected:

- PCR values
- driver names
- firmware details
- kernel configuration

Only boolean state flags are stored.

25.7 Telemetry Storage Model

Telemetry is stored:

- locally
- encrypted
- in a circular buffer
- with a fixed size limit
- with automatic rotation
- with TPM-sealed integrity

If storage is full → oldest entries overwritten.

Telemetry is **not** part of the audit log.

25.8 Telemetry Access Rules

Telemetry can be accessed only:

- by the user
- through Windows Settings

- through enterprise policy tools (local only)

Telemetry cannot be accessed by:

- applications
- scripts
- remote systems
- cloud services
- Copilot itself

Copilot can **read** telemetry only when the user explicitly requests diagnostics.

25.9 Telemetry Export Rules

Export requires:

- explicit user approval
- local export only
- sanitized output
- no identifiers
- no sensitive data
- no raw logs
- no memory dumps

Export formats:

- JSON (sanitized)
- CSV (sanitized)

Enterprises may enforce:

- export encryption
- export retention
- export frequency

25.10 Telemetry Retention Rules

Default retention:

- 30 days
- or until buffer full

Enterprise override:

- shorter retention
- longer retention
- immediate purge on policy change

User override:

- manual purge at any time

Purge must:

- delete all telemetry
- preserve audit logs
- preserve permissions
- preserve sandbox state

25.11 Telemetry Integrity

Telemetry integrity is enforced via:

- TPM-sealed hash
- append-only structure
- rotation integrity checks
- corruption detection

If telemetry is corrupted:

1. Telemetry subsystem resets
2. corrupted buffer is sealed
3. audit log entry is written
4. API continues (telemetry is non-critical)

25.12 Telemetry vs. Audit Log

Telemetry is:

- non-critical
- non-forensic
- non-authoritative
- performance-oriented

Audit log is:

- critical
- forensic
- authoritative
- security-oriented

Telemetry may reference audit log entry IDs but never duplicates content.

25.13 Example Telemetry Entry (Sanitized)

Json

```
{
  "timestamp": "2026-03-13T20:22:00Z",
  "operation": "file.create",
  "duration_ms": 14,
  "sandbox_id": "VBS-INSTANCE-9A1C",
  "syscalls": 3,
  "permission_eval_ms": 2,
  "error": null
}
```

25.14 Example Error Diagnostic Entry

Json

```
{
  "timestamp": "2026-03-13T20:22:10Z",
  "error_code": "PATH_NOT_ALLOWED",
  "subsystem": "PathValidator",
  "sandbox_state": "terminated",
  "last_successful_op": "file.read"
}
```

25.15 Summary

Appendix N defines the complete local-only telemetry and diagnostics model.
Telemetry is strictly offline, sanitized, bounded, deterministic, and non-sensitive.
It supports debugging and performance analysis without compromising privacy, security, or audit integrity.

---

26. Appendix O — Threat Model (STRIDE-Aligned, Full Technical Specification)

This appendix defines the complete threat model for the Local Copilot API using the STRIDE methodology.
The model covers all components: sandbox, permission gateway, system call proxy, audit log, SDK, and baseline enforcement.

The threat model is exhaustive and assumes a capable adversary with local access.

26.1 STRIDE Overview

STRIDE categories:

- **S**poofing
- **T**ampering
- **R**epudiation
- **I**nformation Disclosure
- **D**enial of Service
- **E**levation of Privilege

Each category is analyzed across all system components.

26.2 Threat Surface Overview

Primary components analyzed:

1. Local Execution Layer (sandbox)
2. System Call Proxy Layer (SCPL)
3. Permission Gateway
4. Audit Log + TPM binding
5. SDK
6. Security Baseline Enforcement
7. Network subsystem (optional)
8. Application Trust Model (SAC)
9. Path Safety Model

Each component has unique attack vectors.

26.3 Threat Category: Spoofing

Potential Attacks:

- Fake permission prompts
- Fake application identities
- Fake file paths via symlinks
- Fake sandbox instance IDs
- Fake system call origins
- Fake domain names (network mode)

Mitigations:

- OS-rendered permission UI
- SAC identity verification
- canonical path resolution
- symlink resolution + escape detection
- TPM-sealed sandbox IDs
- SCPL call-origin validation
- HTTPS-only + pinned domain validation

Residual risk: none.

26.4 Threat Category: Tampering

Potential Attacks:

- modifying audit logs
- modifying sandbox memory
- modifying permission store
- modifying SDK schemas
- modifying system call parameters
- modifying baseline configuration
- modifying network requests

Mitigations:

- TPM-sealed audit log chain
- VBS-isolated memory
- encrypted permission store
- SDK schema integrity checks
- SCPL parameter validation
- baseline integrity hashing
- static request construction

Residual risk: physical access.

26.5 Threat Category: Repudiation

Potential Attacks:

- user denies performing an action
- attacker attempts to erase logs
- attacker attempts to forge logs
- attacker attempts to bypass logging

Mitigations:

- append-only TPM-sealed audit log
- hash-chained entries
- sandbox cannot execute without logging
- no silent operations
- no log deletion without trace

Residual risk: none.

26.6 Threat Category: Information Disclosure

Potential Attacks:

- reading system directories
- reading other users' files
- reading protected registry keys
- reading memory outside sandbox
- reading network responses containing sensitive data
- leaking telemetry

Mitigations:

- strict path safety rules
- no registry access
- VBS memory isolation
- text-only, size-limited network responses
- local-only telemetry
- no environment fingerprinting

Residual risk: user-approved file reads.

26.7 Threat Category: Denial of Service

Potential Attacks:

- exhausting sandbox memory
- exhausting telemetry buffer
- exhausting permission store
- forcing repeated sandbox restarts
- corrupting audit log
- breaking baseline integrity

Mitigations:

- fixed memory pools
- circular telemetry buffer
- permission store size limits
- sandbox restart throttling
- audit log corruption detection
- baseline enforcement logic

Residual risk: user intentionally disabling baseline.

26.8 Threat Category: Elevation of Privilege

Potential Attacks:

- launching privileged applications
- interacting with elevated windows
- bypassing SAC
- writing to system directories
- loading DLLs
- spawning processes
- executing scripts
- using network to escalate

Mitigations:

- SAC enforcement
- forbidden system directories
- forbidden executable writes
- no DLL loading
- no process creation
- no script hosts
- HTTPS-only, static network requests
- sandbox integrity enforcement

Residual risk: none.

26.9 Component-Specific Threat Analysis

26.9.1 Sandbox

Threats:

- memory corruption
- system call injection
- escape via NT calls
- JIT or dynamic code execution

Mitigations:

- MGM bounds checking
- SCPL whitelist
- no NT calls
- no executable memory

26.9.2 Permission Gateway

Threats:

- forged permissions
- stale permissions
- wildcard permissions
- permission escalation

Mitigations:

- explicit user approval
- no wildcards
- expiration + max executions
- TPM-sealed store

26.9.3 Audit Log

Threats:

- deletion
- modification
- reordering
- replay attacks

Mitigations:

- hash chain
- TPM sealing
- append-only
- rotation integrity

26.9.4 SDK

Threats:

- schema tampering
- malicious command construction
- bypassing validation

Mitigations:

- schema integrity checks
- command builder restrictions
- local validation

26.9.5 Network Subsystem

Threats:

- exfiltration
- DNS spoofing
- MITM
- dynamic URLs

Mitigations:

- HTTPS-only
- static URLs
- no DNS APIs
- domain pinning

26.10 Threats Outside Scope

The system does **not** defend against:

- physical access attacks
- firmware-level compromise
- kernel-level compromise before boot
- malicious hardware
- user-approved destructive actions

These are explicitly out of scope.

26.11 Threat Model Summary Table

STRIDE	Threat	Mitigation	Residual Risk
S	identity spoofing	SAC, TPM, canonicalization	none
T	tampering	VBS, TPM, SCPL	physical access
R	repudiation	audit log chain	none
I	disclosure	path rules, isolation	user-approved reads
D	DoS	fixed pools, baseline	user disabling baseline
E	privilege escalation	SAC, no exec, no DLL	none

26.12 Summary

Appendix O defines the complete STRIDE-aligned threat model.
The Local Copilot API is designed to resist spoofing, tampering, repudiation, information disclosure, denial of service, and privilege escalation through strict isolation, TPM anchoring, deterministic execution, and minimal attack surface.

---

27. Appendix P — Formal Verification Targets & Non-Bypassability Proof Strategy

This appendix defines the components of the Local Copilot API that must undergo formal verification, the verification methods, and the non-bypassability proof strategy.
The goal is to mathematically guarantee that no operation can escape sandbox boundaries, bypass permission checks, or violate deterministic execution rules.

27.1 Formal Verification Objectives

The verification process must prove:

1. **Non-bypassability** of:
   - sandbox boundaries
   - permission gateway
   - system call whitelist
   - path safety rules
   - SAC trust enforcement
   - baseline enforcement
2. **Determinism** of:
   - runtime execution
   - memory allocation
   - system call behavior
   - error handling
3. **Integrity** of:
   - audit log chain
   - TPM-sealed values
   - sandbox initialization sequence
4. **Non-interference** between:
   - sandbox and host OS
   - sandbox instances
   - user sessions

27.2 Components Subject to Formal Verification

The following components must be formally verified:

1. Deterministic Runtime Engine (DRE)

- state machine correctness
- no nondeterministic transitions
- no hidden states

2. Memory Guard Module (MGM)

- bounds checking
- no pointer arithmetic
- no memory leaks
- no cross-region access

3. System Call Proxy Layer (SCPL)

- strict whitelist enforcement
- parameter validation
- no fallback paths
- no NT call exposure

4. Permission Gateway

- permission matching logic
- expiration logic
- max-execution logic
- revocation logic

5. Path Safety Model

- canonicalization correctness
- traversal detection
- symlink resolution
- forbidden path detection

6. Audit Log Chain

- hash chain correctness
- TPM sealing logic
- rotation integrity

7. Baseline Enforcement Logic

- state machine correctness
- no partial activation
- no degraded mode

27.3 Formal Methods Used

The following formal methods must be applied:

1. Model Checking

Used for:

- state machines
- permission evaluation
- baseline enforcement

Tools:

- CTL/LTL model checkers
- SMT solvers

2. Symbolic Execution

Used for:

- path canonicalization
- system call validation
- memory safety

3. Static Analysis

Used for:

- code integrity
- forbidden API detection
- dead code elimination

4. Formal Specification Languages

Used for:

- defining invariants
- defining pre/post-conditions
- defining safety properties

Languages:

- TLA+
- Z notation
- Alloy

27.4 Non-Bypassability Proof Targets

The following invariants must be proven:

Invariant 1 — No System Call Outside Whitelist

```
∀ call ∈ executed_calls: call ∈ whitelist
```

Invariant 2 — No Operation Without Permission

```
∀ op ∈ executed_ops: permission(op) = true
```

Invariant 3 — No Path Outside Allowed Scope

```
∀ path ∈ resolved_paths: path ∈ allowed_dirs
```

Invariant 4 — No Sandbox Escape

```
sandbox_memory ∩ host_memory = ∅
```

Invariant 5 — No Execution Under Degraded Baseline

```
baseline_state = valid → API_active
baseline_state ≠ valid → API_inactive
```

Invariant 6 — Audit Log Integrity

```
∀ entries: hash(entry[n]) = H(entry[n], hash(entry[n-1]))
```

27.5 Attack Surface Reduction Proof Strategy

The proof strategy ensures:

- no dynamic linking
- no JIT
- no script execution
- no process creation
- no DLL loading
- no raw NT calls
- no device I/O
- no registry modification
- no network sockets

Each forbidden capability is proven unreachable.

27.6 Formal Verification of Determinism

Determinism is proven by:

1. State Machine Equivalence

```
execute(op, state1) = (result, state2)
execute(op, state1) = (result, state2)
```

for all valid states.

2. Memory Allocation Sequence Proof

Memory pool allocation is modeled as a finite automaton with no nondeterministic transitions.

3. System Call Normalization Proof

All system call outputs are normalized to deterministic values.

4. Error Code Determinism

Every error path leads to a single, fixed error code.

27.7 Formal Verification of Permission Model

The permission model must satisfy:

1. No Implicit Permissions

```
permission(op) = false unless explicitly granted
```

2. No Wildcards

```
∀ p: p.scope ∉ wildcard_patterns
```

3. Expiration Enforcement

```
now > p.expires → permission(op) = false
```

4. Max-Execution Enforcement

```
p.remaining = 0 → permission(op) = false
```

5. Revocation Enforcement

```
p.revoked = true → permission(op) = false
```

27.8 Formal Verification of Path Safety

The canonicalization pipeline must satisfy:

1. No Traversal

```
".." ∉ canonical_path
```

2. No Symlink Escape

```
resolve_symlinks(path) ∈ allowed_dirs
```

3. No Device Paths

```
canonical_path ∉ device_paths
```

27.9 Formal Verification of Audit Log Chain

The audit log must satisfy:

1. Hash Chain Integrity

```
entry[n].previous_hash = hash(entry[n-1])
```

2. TPM Sealing

```
unseal(entry[n].integrity_hash) = hash(entry[n])
```

3. Append-Only

```
∀ entries: no deletion, no modification
```

27.10 Non-Bypassability Proof Summary

The system is proven non-bypassable if:

- sandbox cannot escape
- SCPL cannot be bypassed
- permissions cannot be forged
- paths cannot escape allowed scope
- audit logs cannot be modified
- baseline cannot be degraded silently
- network cannot be used for exfiltration
- SAC cannot be bypassed

All proofs must be machine-checked.

27.11 Summary

Appendix P defines the formal verification targets and the non-bypassability proof strategy. The system must be mathematically proven to enforce sandbox isolation, permission correctness, deterministic execution, and audit integrity.
No component may rely on heuristics or trust assumptions — only verifiable guarantees.

---

28. Appendix Q — Compliance Mapping

(GDPR, EU Cybersecurity Act, NIST 800-53, ISO 27001)

This appendix maps the Local Copilot API architecture, controls, and safeguards to major regulatory and security frameworks.
The mapping is technical, control-level, and strictly scoped to the system as designed.

28.1 GDPR Compliance Mapping

The Local Copilot API is designed to minimize personal data processing.
All processing is local, deterministic, and user-controlled.

28.1.1 GDPR Principles Mapping

GDPR Principle	Implementation in Local Copilot API
Lawfulness, fairness, transparency	Explicit permission prompts; no hidden processing
Purpose limitation	No cloud processing; no secondary use
Data minimization	No telemetry export; no identifiers stored
Accuracy	Deterministic execution; no heuristic inference
Storage limitation	Telemetry retention limits; user purge
Integrity & confidentiality	VBS isolation, TPM sealing, SAC enforcement
Accountability	TPM-sealed audit logs

28.1.2 GDPR Articles Mapping (Technical)

- **Art. 5** — Data minimization → No cloud, no sync, no identifiers
- **Art. 6** — Lawful basis → Explicit user approval for each operation
- **Art. 25** — Privacy by design → Local-only, sandboxed, deterministic
- **Art. 30** — Records of processing → Audit log chain
- **Art. 32** — Security of processing → VBS, TPM, SAC, baseline enforcement
- **Art. 33/34** — Breach notification → Local system responsibility
- **Art. 35** — DPIA → Threat model + formal verification

28.2 EU Cybersecurity Act Mapping

The Local Copilot API aligns with the **EU Cybersecurity Act** requirements for high-assurance ICT products.

28.2.1 High-Assurance Requirements Mapping

Requirement	Implementation
Secure by design	Sandbox, SCPL, permission gateway
Secure by default	Network disabled, no scripts, no exec
Vulnerability handling	Audit log, baseline enforcement
Transparency	Deterministic behavior, no hidden flows
Strong isolation	VBS, MGM, SCPL
Cryptographic integrity	TPM-sealed logs, integrity checks

28.2.2 Assurance Level Alignment

The architecture aligns with **EUCC High** (EU Cybersecurity Certification Scheme) due to:

- hardware-anchored trust
- deterministic execution
- formal verification targets
- non-bypassability guarantees

28.3 NIST 800-53 Compliance Mapping

The Local Copilot API aligns with NIST 800-53 Rev. 5 security controls.

28.3.1 Relevant Control Families

Control Family	Implementation
AC — Access Control	Permission gateway, SAC
AU — Audit & Accountability	TPM-sealed audit log
CM — Configuration Management	Baseline enforcement
IA — Identification & Authentication	SAC trust model
MP — Media Protection	No external storage
PL — Security Planning	Threat model, formal verification
SC — System & Communications Protection	Sandbox, SCPL
SI — System Integrity	VBS, TPM, integrity checks

28.3.2 Key Controls Mapping

- **AC-3** — Access enforcement → Permission gateway
- **AC-6** — Least privilege → No privileged operations
- **AU-2** — Audit events → Full operation logging
- **AU-9** — Protection of audit info → TPM sealing
- **SC-3** — Security function isolation → Sandbox
- **SC-7** — Boundary protection → SCPL
- **SC-39** — Process isolation → VBS
- **SI-7** — Software integrity → SAC + integrity checks

28.4 ISO 27001 Compliance Mapping

ISO 27001 Annex A controls are mapped to system safeguards.

28.4.1 Annex A Control Mapping

ISO 27001 Control	Implementation
A.5 — Information security policies	Deterministic, local-only design
A.8 — Asset management	No external data flows
A.9 — Access control	Permission model
A.10 — Cryptography	TPM sealing
A.11 — Physical security	Out of scope (OS responsibility)
A.12 — Operations security	Sandbox isolation
A.13 — Communications security	Network disabled by default
A.14 — System acquisition	Formal verification
A.16 — Incident management	Audit log + baseline enforcement
A.18 — Compliance	GDPR alignment

28.5 Cross-Framework Summary

Framework	Compliance Mechanism
GDPR	Local-only, explicit permissions, no identifiers
EU Cybersecurity Act	High-assurance architecture, TPM, VBS
NIST 800-53	Strong isolation, auditability, access control
ISO 27001	Cryptography, access control, operations security

28.6 Non-Compliance Areas (Explicit)

The system **does not** cover:

- physical security (ISO 27001 A.11)
- organizational governance (ISO 27001 A.5.1)
- cloud data processing (GDPR Art. 44–49)
- cross-border data transfer
- multi-tenant environments

These are explicitly out of scope.

28.7 Summary

Appendix Q provides a full compliance mapping across GDPR, EU Cybersecurity Act, NIST 800-53, and ISO 27001.
The Local Copilot API architecture aligns with high-assurance security and privacy requirements through deterministic execution, sandbox isolation, TPM anchoring, explicit permissions, and strict non-bypassability guarantees.

---

30. Appendix S — Full API Surface (Operations, Schemas, Constraints)

This appendix defines the **complete, authoritative API surface** of the Local Copilot API.
Only the operations listed here exist.
No hidden, dynamic, or undocumented operations are allowed.

All operations must:

- be deterministic
- be sandbox-safe
- require explicit permission
- follow strict schemas
- be validated before execution
- be logged in the audit log

30.1 API Surface Overview

The API surface is divided into six categories:

1. File Operations
2. Directory Operations
3. Application Operations
4. Workflow Operations
5. Network Operations (optional)
6. System Information Operations

Each category contains a fixed set of operations.

30.2 File Operations

Allowed operations:

- `file.create`
- `file.read`
- `file.write`
- `file.append`
- `file.rename`
- `file.delete`
- `file.info`

Forbidden:

- writing binary data
- writing executable files
- modifying system files
- accessing forbidden paths

30.2.1 Schema: file.create

Json

```
{
  "operation": "file.create",
  "path": "string",
  "content": "string"
}
```

Constraints:

- path must be canonical
- content must be UTF-8 text
- file must not already exist (unless overwrite flag is explicit)

30.2.2 Schema: file.read

Json

```
{
  "operation": "file.read",
  "path": "string",
  "max_bytes": 1048576
}
```

Constraints:

- max 1 MB
- text-only normalization

30.2.3 Schema: file.write

Json

```
{
  "operation": "file.write",
  "path": "string",
  "content": "string"
}
```

Constraints:

- overwrites file
- text-only
- no executable extensions

30.2.4 Schema: file.append

Json

```
{
  "operation": "file.append",
  "path": "string",
  "content": "string"
}
```

Constraints:

- text-only
- file must exist

30.2.5 Schema: file.rename

Json

```
{
  "operation": "file.rename",
  "source": "string",
  "target": "string"
}
```

Constraints:

- both paths must be canonical
- no cross-directory moves into forbidden areas

30.2.6 Schema: file.delete

Json

```
{
  "operation": "file.delete",
  "path": "string"
}
```

Constraints:

- no system files
- no protected directories

30.2.7 Schema: file.info

Json

```
{
  "operation": "file.info",
  "path": "string"
}
```

Returns:

- size
- timestamps
- attributes

30.3 Directory Operations

Allowed operations:

- `directory.create`
- `directory.delete`
- `directory.list`
- `directory.info`

Forbidden:

- recursive delete
- system directory access

30.3.1 Schema: directory.create

Json

```
{
  "operation": "directory.create",
  "path": "string"
}
```

30.3.2 Schema: directory.delete

Json

```
{
  "operation": "directory.delete",
  "path": "string"
}
```

Constraints:

- directory must be empty

30.3.3 Schema: directory.list

Json

```
{
  "operation": "directory.list",
  "path": "string"
}
```

Returns:

- sorted list of entries

30.3.4 Schema: directory.info

Json

```
{
  "operation": "directory.info",
  "path": "string"
}
```

30.4 Application Operations

Allowed operations:

- `app.launch`
- `app.close`
- `app.focus`
- `app.list_open`

Forbidden:

- interacting with elevated apps
- sending keystrokes
- automating UI

30.4.1 Schema: app.launch

Json

```
{
  "operation": "app.launch",
  "application": "string"
}
```

Constraints:

- SAC-approved
- signed
- non-elevated

30.4.2 Schema: app.close

Json

```
{
  "operation": "app.close",
  "window_id": "string"
}
```

30.4.3 Schema: app.focus

Json

```
{
  "operation": "app.focus",
  "window_id": "string"
}
```

30.4.4 Schema: app.list_open

Json

```
{
  "operation": "app.list_open"
}
```

30.5 Workflow Operations

Allowed operations:

- `workflow.batch_rename`
- `workflow.organize`
- `workflow.extract_text`

All workflows must be deterministic.

30.5.1 Schema: workflow.batch_rename

Json

```
{
  "operation": "workflow.batch_rename",
  "directory": "string",
  "pattern": "string",
  "replacement": "string"
}
```

30.5.2 Schema: workflow.organize

Json

```
{
  "operation": "workflow.organize",
  "directory": "string",
  "rules": [ ... ]
}
```

30.5.3 Schema: workflow.extract_text

Json

```
{
  "operation": "workflow.extract_text",
  "path": "string"
}
```

30.6 Network Operations (Optional)

Allowed operations:

- `network.request`

30.6.1 Schema: network.request

Json

```
{
  "operation": "network.request",
  "domain": "string",
  "path": "string",
  "method": "GET | POST",
  "payload": "string | null"
}
```

Constraints:

- HTTPS only
- static URLs
- text-only responses
- max 1 MB

30.7 System Information Operations

Allowed operations:

- `system.disk_info`
- `system.memory_info`
- `system.version_info`

30.7.1 Schema: system.disk_info

Json

```
{
  "operation": "system.disk_info",
  "path": "string"
}
```

30.7.2 Schema: system.memory_info

Json

```
{
  "operation": "system.memory_info"
}
```

30.7.3 Schema: system.version_info

Json

```
{
  "operation": "system.version_info"
}
```

30.8 Global Constraints

All operations must satisfy:

- canonical path rules
- permission model
- sandbox isolation
- deterministic execution
- audit logging
- baseline enforcement
- SAC trust model

No operation may bypass these constraints.

30.9 Summary

Appendix S defines the **complete, fixed API surface** of the Local Copilot API.
No additional operations exist.
All operations follow strict schemas, deterministic rules, and sandbox-safe constraints.

---

31. Appendix T — SDK Architecture

(Client-Side Enforcement, Builders, Validators)

This appendix defines the architecture of the Local Copilot API SDK.
The SDK is responsible for:

- constructing valid commands
- enforcing client-side constraints
- validating schemas before execution
- preventing malformed or unsafe requests
- ensuring deterministic request construction
- integrating with permission prompts
- providing a stable, typed interface

The SDK does **not** execute operations.
It only constructs and validates them.

31.1 SDK Objectives

The SDK must:

- prevent invalid commands from reaching the sandbox
- enforce schemas at build time
- enforce deterministic ordering of fields
- prevent injection of unsupported operations
- prevent bypass of permission model
- prevent construction of unsafe paths
- normalize all inputs
- guarantee compatibility with formal verification

The SDK is a **safety layer**, not a convenience layer.

31.2 SDK Architecture Overview

The SDK consists of:

1. **Command Builders**
2. **Validators**
3. **Canonicalizers**
4. **Permission Integrators**
5. **Schema Registry**
6. **Transport Layer (Local IPC)**

Each component enforces a different constraint.

31.3 Command Builders

Builders construct commands using a strict, typed interface.

Example (file.create):

```
FileCreateBuilder
.path("C:\\Users\\User\\Documents\\file.txt")
.content("Hello")
.build()
```

Builder rules:

- no dynamic fields
- no optional fields unless defined in schema
- no hidden fields
- no metadata injection
- deterministic field ordering

Builders cannot:

- modify paths
- auto-correct invalid input
- infer missing fields

31.4 Validators

Validators enforce:

- schema correctness
- type correctness
- path safety
- operation constraints
- size limits
- forbidden extension rules
- deterministic ordering

Validation occurs **before** sending the command to the sandbox.

Validation Pipeline:

```
1. Schema validation
2. Type validation
3. Path canonicalization
4. Path safety validation
5. Operation-specific constraints
6. Permission pre-check
7. Deterministic serialization
```

If any step fails → exception.

31.5 Canonicalizers

Canonicalizers ensure:

- slash normalization
- absolute path conversion
- environment variable expansion
- symlink resolution (client-side preview only)
- removal of redundant segments

Canonicalizers do **not**:

- resolve symlinks fully (sandbox re-validates)
- allow traversal
- allow NT paths
- allow UNC paths

31.6 Permission Integrators

The SDK integrates with the permission system by:

- checking if permission exists
- checking expiration
- checking max-execution count
- triggering permission prompts when needed

Permission prompts are:

- OS-rendered
- non-spoofable
- non-skippable
- non-automatable

SDK cannot:

- auto-approve permissions
- store permissions
- modify permissions

31.7 Schema Registry

The SDK contains a **static schema registry**.

Properties:

- immutable
- versioned
- formally verified
- no dynamic schema loading
- no remote schema updates

Each operation has:

- required fields
- optional fields
- field types
- constraints
- deterministic serialization rules

Example registry entry:

```
operation: "file.create"

fields:
  path: string (canonical)
  content: string (utf8)

constraints:
  max_size: 1MB
  no_executable_extensions
```

31.8 Transport Layer (Local IPC)

The SDK communicates with the sandbox via:

- local IPC
- authenticated channel
- deterministic message format
- no network
- no cloud
- no remote endpoints

Transport guarantees:

- no message reordering
- no message mutation
- no hidden metadata
- no compression
- no streaming

31.9 Deterministic Serialization

All commands must serialize deterministically:

- sorted keys
- UTF-8 encoding
- no whitespace variance
- no optional formatting
- no timestamps
- no random IDs (sandbox generates IDs)

Example:
Json

```
{
  "content": "Hello",
  "operation": "file.create",
  "path": "C:\\Users\\User\\Documents\\file.txt"
}
```

31.10 SDK Error Model

Errors must:

- use fixed error codes
- never expose stack traces
- never expose internal state
- never expose memory addresses
- never expose OS-level errors directly

Example:

```
SDK_ERROR_INVALID_PATH
SDK_ERROR_SCHEMA_MISMATCH
SDK_ERROR_PERMISSION_REQUIRED
SDK_ERROR_CANONICALIZATION_FAILED
```

31.11 SDK Safety Guarantees

The SDK guarantees:

- no malformed commands reach sandbox
- no unsafe paths reach sandbox
- no unsupported operations reach sandbox
- no dynamic or hidden fields
- no nondeterministic behavior
- no bypass of permission model
- no bypass of schema constraints

The SDK is a **mathematically constrained interface**.

31.12 Summary

Appendix T defines the complete SDK architecture.
The SDK enforces schemas, validates inputs, canonicalizes paths, integrates permissions, and serializes commands deterministically.
It prevents malformed, unsafe, or unauthorized commands from ever reaching the sandbox.

---

32. Appendix U — End-to-End Execution Flow

(From User Intent to Sandbox Completion)

This appendix defines the complete, deterministic execution flow of the Local Copilot API.
It describes every stage from user intent to sandbox execution, audit logging, and final response.
No hidden steps, no background processes, no nondeterministic behavior.

32.1 High-Level Flow Overview

The full execution pipeline consists of:

1. User Intent Parsing
2. SDK Command Construction
3. Client-Side Validation
4. Permission Evaluation
5. Canonicalization & Safety Checks
6. Deterministic Serialization
7. Local IPC Transport
8. Sandbox Intake Validation
9. System Call Proxy Execution
10. Result Normalization
11. Audit Logging
12. Response Return

Every step is mandatory.

32.2 Step 1 — User Intent Parsing

The Copilot model interprets user intent and maps it to:

- a specific operation
- required parameters
- optional parameters
- constraints

The model **cannot**:

- infer missing paths
- guess unsafe defaults
- auto-approve permissions
- bypass SDK validation

32.3 Step 2 — SDK Command Construction

The SDK constructs a command using:

- typed builders
- static schemas
- deterministic field ordering

Example:

```
FileCreateBuilder
.path("C:\\Users\\User\\Documents\\note.txt")
.content("Hello")
.build()
```

No dynamic fields.
No hidden metadata.

32.4 Step 3 — Client-Side Validation

Validation pipeline:

```
schema → types → canonicalization → path safety → constraints → permission
pre-check
```

If any step fails → command rejected.

32.5 Step 4 — Permission Evaluation

The SDK checks:

- explicit permission exists
- permission not expired
- max-execution count not exceeded
- permission not revoked

If permission missing → OS-rendered prompt.

SDK cannot auto-approve.

32.6 Step 5 — Canonicalization & Safety Checks

Performed by SDK and re-validated by sandbox.

Rules enforced:

- no traversal
- no symlink escape
- no system directories
- no UNC paths
- no NT paths
- no device paths
- no executable extensions

Canonicalization must be deterministic.

32.7 Step 6 — Deterministic Serialization

Command is serialized with:

- sorted keys
- UTF-8 encoding
- no whitespace variance
- no timestamps
- no random IDs

Sandbox generates request ID.

32.8 Step 7 — Local IPC Transport

Transport layer guarantees:

- local-only
- authenticated
- no network
- no compression
- no streaming
- no metadata injection

Messages cannot be reordered or mutated.

32.9 Step 8 — Sandbox Intake Validation

Sandbox performs:

1. schema validation
2. canonicalization
3. path safety
4. permission validation
5. baseline validation
6. SAC validation
7. memory safety pre-check

If any step fails → sandbox aborts operation.

32.10 Step 9 — System Call Proxy Execution (SCPL)

The SCPL:

- maps operation to whitelisted system calls
- validates parameters
- enforces deterministic behavior
- blocks forbidden calls
- blocks NT calls
- blocks device I/O
- blocks registry access
- blocks process creation

If a forbidden call is attempted → sandbox termination.

32.11 Step 10 — Result Normalization

Sandbox normalizes:

- timestamps → UTC
- paths → canonical
- errors → deterministic codes
- file content → UTF-8
- directory listings → sorted

No OS-dependent variance allowed.

32.12 Step 11 — Audit Logging

Audit log entry includes:

- timestamp
- operation
- parameters (sanitized)
- result
- sandbox ID
- integrity hash
- previous hash

Entry is:

- hash-chained
- TPM-sealed
- append-only

No silent logging.
No log suppression.

32.13 Step 12 — Response Return

The sandbox returns:

- normalized result
- deterministic error (if any)
- request ID

SDK delivers result to Copilot.
Copilot formats final user-visible output.

32.14 Full Execution Flow Diagram (Linear)

```
User Intent
↓
Intent Parser
↓
SDK Builder
↓
SDK Validator
↓
Permission Gateway
↓
Canonicalization
↓
Deterministic Serialization
↓
Local IPC
↓
Sandbox Intake Validator
↓
SCPL (Whitelisted Syscalls)
↓
Deterministic Result Normalization
↓
Audit Log (TPM-Sealed)
↓
SDK Response
↓
Copilot Output
```

32.15 Summary

Appendix U defines the complete, deterministic, end-to-end execution flow.
Every stage is explicit, validated, sandboxed, logged, and non-bypassable.
No hidden steps, no background tasks, no nondeterministic behavior.

---

33. Appendix V — Sandbox State Machine (Formal Definition)

This appendix defines the **formal state machine** governing the sandbox lifecycle.
The sandbox is a deterministic, single-threaded, VBS-isolated execution environment with strict transitions and no implicit states.

The state machine is **closed**, **finite**, and **non-bypassable**.

33.1 State Machine Overview

The sandbox has exactly **seven** states:

1. **S0 — Uninitialized**
2. **S1 — Baseline Validation**
3. **S2 — Initialization**
4. **S3 — Ready**
5. **S4 — Executing**
6. **S5 — Terminating**
7. **S6 — Terminated**

No other states exist.

33.2 State Transition Diagram (Linear Form)

```
S0 → S1 → S2 → S3 → S4 → S3 → S5 → S6
```

Loops allowed:

- S3 → S4 → S3 (multiple operations)
- S3 → S5 (shutdown)
- S4 → S5 (failure)

No backward transitions except S4 → S3.

33.3 State Definitions

S0 — Uninitialized

Sandbox memory not allocated.
No operations allowed.

Entry conditions:

- system boot
- API activation request

Exit condition:

- begin baseline validation

S1 — Baseline Validation

Validates:

- TPM
- Secure Boot
- VBS
- HVCI
- SAC
- unsigned drivers
- baseline integrity hash

If any check fails → S6 (Terminated).

Exit condition:

- all checks pass → S2

S2 — Initialization

Actions:

- allocate memory pools
- load deterministic runtime engine
- initialize SCPL
- initialize MGM
- initialize schema registry
- generate sandbox ID
- verify integrity hashes

Failure → S6.

Success → S3.

S3 — Ready

Sandbox is idle and ready for commands.

Allowed actions:

- accept command
- validate command
- transition to S4

Forbidden:

- background tasks
- asynchronous operations
- implicit transitions

S4 — Executing

Sandbox is executing exactly **one** operation.

Actions:

- SCPL system calls
- deterministic runtime execution
- memory-safe operations
- result normalization
- audit logging

Exit conditions:

- success → S3
- recoverable error → S3
- critical error → S5

S5 — Terminating

Sandbox is shutting down.

Actions:

- flush audit log
- release memory pools
- invalidate sandbox ID
- write termination reason

Exit → S6.

S6 — Terminated

Sandbox is inactive.

Allowed actions:

- none

Recovery requires:

- full restart
- full baseline validation
- full initialization

No partial recovery.

33.4 Formal Transition Table

Current	Event	Next	Notes
S0	start	S1	begin baseline validation
S1	baseline_ok	S2	all checks passed
S1	baseline_fail	S6	immediate termination
S2	init_ok	S3	sandbox ready
S2	init_fail	S6	cannot initialize
S3	command_received	S4	begin execution
S4	exec_success	S3	return to ready
S4	exec_error_recoverable	S3	deterministic error
S4	exec_error_critical	S5	terminate
S3	shutdown	S5	user/system request
S5	terminate_complete	S6	final state

33.5 Forbidden Transitions

The following transitions are **impossible** and must be proven unreachable:

- S4 → S2
- S4 → S1
- S3 → S1
- S5 → S3
- S6 → S3
- S6 → S4
- S4 → S4 (no re-entrancy)
- S3 → S3 (no idle loops)

Any attempt to force these transitions → sandbox termination.

33.6 Deterministic Execution Guarantees

The state machine enforces:

- no concurrency
- no race conditions
- no re-entrant execution
- no background threads
- no hidden states
- no nondeterministic transitions

Every transition is explicit and logged.

33.7 Failure Handling

Critical failures (baseline degradation, SCPL violation, MGM violation):

```
S4 → S5 → S6
```

Recoverable failures (invalid path, permission denied):

```
S4 → S3
```

No silent failures.

33.8 Formal Invariants

Invariant 1 — Single Operation

```
state = S4 → executing_ops = 1
```

Invariant 2 — No Execution Without Baseline

```
baseline_valid = false → state ∉ {S3, S4}
```

Invariant 3 — No Execution After Termination

```
state = S6 → executing_ops = 0
```

Invariant 4 — No Hidden Transitions

```
∀ transitions: must match transition table
```

Invariant 5 — Deterministic State

```
same_input, same_state → same_next_state
```

33.9 Summary

Appendix V defines the formal sandbox state machine.
It is finite, deterministic, non-bypassable, and fully auditable.
No implicit states, no hidden transitions, no concurrency, no nondeterminism.

---

34. Appendix W — Memory Model

(Static Pools, Regions, Deterministic Allocation)

This appendix defines the complete memory model of the Local Copilot API sandbox.
The memory model is **static**, **deterministic**, **non-executable**, and **fully isolated** via VBS.
No dynamic allocation, no heap growth, no nondeterministic garbage collection.

34.1 Memory Model Objectives

The memory model must:

- guarantee deterministic allocation
- prevent memory corruption
- prevent cross-region access
- prevent executable memory
- prevent shared memory
- prevent race conditions
- support formal verification
- support deterministic teardown

No dynamic or unpredictable behavior is allowed.

34.2 Memory Regions Overview

The sandbox memory is divided into **five fixed regions**:

1. **R0 — Runtime Region**
2. **R1 — Command Buffer Region**
3. **R2 — Working Memory Region**
4. **R3 — SCPL Buffer Region**
5. **R4 — Log Buffer Region**

Each region has a fixed size and fixed purpose.

34.3 Region Definitions

R0 — Runtime Region

Contains:

- deterministic runtime engine
- state machine
- schema registry
- SCPL dispatch table

Properties:

- read-only
- integrity-checked
- non-executable

R1 — Command Buffer Region

Contains:

- deserialized command
- canonicalized paths
- validated parameters

Properties:

- fixed size
- cleared after each operation
- no pointers to other regions

R2 — Working Memory Region

Contains:

- temporary buffers
- intermediate results
- normalized outputs

Properties:

- fixed size
- zeroed before and after use
- no persistent data

R3 — SCPL Buffer Region

Contains:

- system call parameters
- system call return values

Properties:

- strictly typed
- normalized
- no raw handles
- no kernel pointers

R4 — Log Buffer Region

Contains:

- pre-hashed audit log entry
- TPM-sealed integrity block

Properties:

- append-only
- fixed size
- flushed after each operation

34.4 Deterministic Allocation Model

The sandbox uses **static pool allocation**:

```
Pool A → Pool B → Pool C → Pool A → ...
```

Rules:

- no malloc
- no free
- no fragmentation
- no dynamic resizing
- no unpredictable reuse

Allocation order is fixed and cyclic.

34.5 Memory Safety Guarantees

The memory model enforces:

- bounds checking on every access
- no pointer arithmetic
- no raw pointers exposed
- no cross-region access
- no aliasing
- no shared memory
- no executable pages

If any violation occurs → sandbox termination.

34.6 Zeroization Rules

Before and after each operation:

- R1, R2, R3 are fully zeroed
- R4 is flushed and zeroed
- R0 remains read-only

Zeroization is deterministic and mandatory.

34.7 Memory Integrity Verification

At initialization:

- all regions hashed
- runtime region verified
- SCPL table verified
- schema registry verified

During execution:

- region boundaries enforced
- region access patterns validated
- SCPL parameters validated

After execution:

- working memory wiped
- command buffer wiped
- SCPL buffer wiped

34.8 Forbidden Memory Behaviors

The sandbox **cannot**:

- allocate memory dynamically
- grow memory pools
- shrink memory pools
- map files into memory
- allocate executable memory
- share memory with host
- use JIT compilation
- use garbage collection
- use stack unwinding
- use recursion (stack depth fixed)

All forbidden behaviors are blocked at compile-time and runtime.

34.9 Memory Access Rules

Allowed:

- fixed-offset reads
- fixed-offset writes
- region-bounded operations
- deterministic buffer operations

Forbidden:

- pointer arithmetic
- variable-length buffers
- unbounded loops
- dynamic indexing
- raw handle storage

34.10 Memory Exhaustion Handling

If any region exceeds its capacity:

```
→ immediate sandbox termination
→ audit log entry
→ API deactivation
```

No fallback.
No partial execution.
No retry.

34.11 Formal Memory Invariants

Invariant 1 — Region Isolation

```
∀ regions Ri, Rj: i ≠ j → Ri ∩ Rj = ∅
```

Invariant 2 — Deterministic Allocation

```
alloc(n) = fixed_sequence[k]
```

Invariant 3 — No Executable Memory

```
∀ pages: executable = false
```

Invariant 4 — Zeroization

```
post_op(R1,R2,R3,R4) = 0
```

Invariant 5 — No Cross-Region Access

```
access(addr) → addr ∈ region_of(op)
```

34.12 Summary

Appendix W defines the complete memory model of the sandbox.
Memory is static, deterministic, isolated, non-executable, and formally verifiable.
No dynamic allocation, no shared memory, no nondeterministic behavior.

---

35. Appendix X — System Call Normalization Layer

(Deterministic Output Mapping)

This appendix defines the **System Call Normalization Layer (SCNL)**, which ensures that all system call outputs are deterministic, sanitized, and independent of OS-specific behavior. Normalization is mandatory for every whitelisted system call.

The SCNL is part of the SCPL (System Call Proxy Layer) but operates as a distinct, strictly defined stage.

35.1 Objectives of the Normalization Layer

The SCNL must:

- eliminate OS-dependent variability
- eliminate nondeterministic values
- normalize timestamps
- normalize file attributes
- normalize error codes
- normalize directory listings
- normalize system information
- ensure deterministic return structures

No raw system call output is ever exposed.

35.2 Normalization Pipeline

Every system call passes through:

```
raw_output
↓
sanitize()
↓
normalize_types()
↓
normalize_values()
↓
normalize_structure()
↓
deterministic_output
```

Each stage is mandatory.

35.3 Type Normalization

All values must be normalized to:

- UTF-8 strings
- integers
- booleans
- fixed-format timestamps
- fixed-format enumerations

Forbidden:

- raw handles
- pointers
- OS-specific structures
- variable-length binary blobs

35.4 Timestamp Normalization

All timestamps must be:

- converted to UTC
- formatted as ISO-8601
- stripped of sub-millisecond precision
- independent of system locale

Example:

```
2026-03-13T20:44:00Z
```

Forbidden:

- local time
- locale-dependent formats
- high-precision timers
- relative timestamps

35.5 File Attribute Normalization

Raw attributes (e.g., Windows FILE_ATTRIBUTE flags) are mapped to deterministic fields:
Json

```
{
  "readonly": true/false,
  "hidden": true/false,
  "system": true/false,
  "directory": true/false,
  "archive": true/false
}
```

Forbidden:

- OS-specific flags
- device attributes
- reparse point metadata

35.6 Error Normalization

Raw OS error codes (e.g., Win32 errors) are mapped to deterministic API error codes.

Example mapping:

Win32 Error	API Error
ERROR_ACCESS_DENIED	PERMISSION_DENIED
ERROR_FILE_NOT_FOUND	FILE_NOT_FOUND
ERROR_PATH_NOT_FOUND	PATH_NOT_ALLOWED
ERROR_INVALID_HANDLE	INVALID_OPERATION
ERROR_SHARING_VIOLATION	FILE_LOCKED

Forbidden:

- exposing raw Win32 codes
- exposing NTSTATUS codes
- exposing kernel error messages

35.7 Directory Listing Normalization

Directory listings must be:

- sorted lexicographically
- filtered for forbidden entries
- normalized to UTF-8
- stripped of system-specific metadata

Each entry:
Json

```
{
  "name": "string",
  "is_directory": true/false,
  "size": integer,
  "modified": "ISO-8601"
}
```

Forbidden:

- unsorted results
- locale-dependent sorting
- raw WIN32_FIND_DATA structures

35.8 Path Normalization

Paths must be:

- canonical
- absolute
- using backslashes
- stripped of trailing slashes
- resolved for symlinks (sandbox-side)

Example:

```
C:\Users\User\Documents\Report2.txt
```

Forbidden:

- forward slashes
- UNC paths
- NT paths
- device paths

35.9 System Information Normalization

System information must be normalized to:
Json

```
{
  "os_version": "Windows 11",
  "build": 99999,
  "architecture": "x64"
}
```

Forbidden:

- kernel version
- driver list
- firmware details
- hardware identifiers

35.10 Network Response Normalization (Optional)

If network mode is enabled:

- responses must be text-only
- responses must be UTF-8
- responses must be ≤ 1 MB
- headers must be stripped
- cookies must be removed
- redirects must be blocked

Forbidden:

- binary responses
- HTML with active content
- compressed responses

35.11 System Call Output Structure

Every normalized output must follow:
Json

```
{
  "success": true/false,
  "result": { ... },
  "error": null | { "code": "...", "details": "..." },
  "timestamp": "ISO-8601",
  "request_id": "GUID"
}
```

No additional fields allowed.

35.12 Deterministic Mapping Rules

The SCNL must ensure:

- same input → same output
- no OS variance
- no locale variance
- no timing variance
- no hardware variance

Normalization is part of formal verification.

35.13 Forbidden Behaviors

The SCNL must not:

- expose raw system call output
- expose kernel structures
- expose memory addresses
- expose handles
- expose OS-specific error messages
- expose nondeterministic values

Any violation → sandbox termination.

35.14 Summary

Appendix X defines the System Call Normalization Layer.
It ensures that all system call outputs are deterministic, sanitized, and independent of OS-specific behavior.
Normalization is mandatory, non-bypassable, and formally verifiable.

---

36. Appendix Y — Error Model

(Deterministic Codes, Categories, Recovery Rules)

This appendix defines the complete error model of the Local Copilot API.
All errors must be:

- deterministic
- categorized
- sanitized
- non-leaking
- non-ambiguous
- formally verifiable

No raw OS errors, no stack traces, no nondeterministic messages.

36.1 Error Model Objectives

The error model must:

- provide deterministic error codes
- prevent information leakage
- support formal verification
- support audit logging
- support safe recovery
- avoid exposing OS internals
- avoid nondeterministic behavior

Errors are part of the API contract.

36.2 Error Structure

Every error has the following structure:
Json

```
{
  "code": "STRING",
  "category": "STRING",
  "details": "STRING (sanitized, optional)"
}
```

Mandatory fields:

- `code`
- `category`

Optional:

- `details` (never contains sensitive data)

36.3 Error Categories

The system defines **seven** error categories:

1. **PATH_ERROR**
2. **PERMISSION_ERROR**
3. **VALIDATION_ERROR**
4. **EXECUTION_ERROR**
5. **SYSTEM_ERROR**
6. **NETWORK_ERROR**
7. **SANDBOX_ERROR**

No other categories exist.

36.4 Deterministic Error Codes

Each category contains fixed error codes.

36.4.1 PATH_ERROR

- `PATH_NOT_ALLOWED`
- `PATH_NOT_FOUND`
- `PATH_SYMLINK_ESCAPE`
- `PATH_FORBIDDEN_EXTENSION`
- `PATH_CANONICALIZATION_FAILED`

36.4.2 PERMISSION_ERROR

- `PERMISSION_REQUIRED`
- `PERMISSION_DENIED`
- `PERMISSION_EXPIRED`
- `PERMISSION_REVOKED`
- `PERMISSION_MAX_EXECUTIONS_REACHED`

36.4.3 VALIDATION_ERROR

- `SCHEMA_MISMATCH`
- `INVALID_OPERATION`
- `INVALID_PARAMETER`
- `INVALID_CONTENT`
- `SIZE_LIMIT_EXCEEDED`

36.4.4 EXECUTION_ERROR

- `FILE_LOCKED`
- `FILE_IN_USE`
- `DIRECTORY_NOT_EMPTY`
- `APPLICATION_NOT_FOUND`
- `APPLICATION_NOT_TRUSTED`

36.4.5 SYSTEM_ERROR

- `BASELINE_INVALID`
- `SAC_DISABLED`
- `TPM_UNAVAILABLE`
- `VBS_UNAVAILABLE`
- `INTEGRITY_CHECK_FAILED`

36.4.6 NETWORK_ERROR (optional)

- `NETWORK_DISABLED`
- `DOMAIN_NOT_ALLOWED`
- `HTTPS_REQUIRED`
- `RESPONSE_TOO_LARGE`
- `UNSAFE_CONTENT`

36.4.7 SANDBOX_ERROR

- `SANDBOX_TERMINATED`
- `SCPL_VIOLATION`
- `MEMORY_VIOLATION`
- `FORBIDDEN_SYSCALL`
- `NONDETERMINISTIC_BEHAVIOR_DETECTED`

36.5 Error Determinism Rules

Errors must be:

- reproducible
- independent of OS locale
- independent of OS version
- independent of hardware
- independent of timing

Example:

```
same invalid path → same error code
```

36.6 Error Sanitization Rules

Errors must **not** contain:

- file contents
- user data
- system identifiers
- registry values
- kernel messages
- raw OS error codes
- stack traces
- memory addresses

Only safe, deterministic information is allowed.

36.7 Recovery Rules

36.7.1 Recoverable Errors

Recoverable errors return the sandbox to **S3 (Ready)**:

- PATH_ERROR
- PERMISSION_ERROR
- VALIDATION_ERROR
- EXECUTION_ERROR
- NETWORK_ERROR

Recovery steps:

1. zeroize working memory
2. flush log buffer
3. return to Ready state

36.7.2 Non-Recoverable Errors

Non-recoverable errors transition to:

```
S4 → S5 → S6
```

These include:

- SYSTEM_ERROR
- SANDBOX_ERROR

Examples:

- baseline invalid
- SCPL violation
- memory violation
- forbidden syscall
- integrity failure

36.8 Error Logging Rules

Every error must generate an audit log entry containing:

- error code
- category
- sanitized details
- timestamp
- sandbox ID
- request ID
- integrity hash

No error may bypass logging.

36.9 Error Propagation Rules

Errors propagate:

```
SCPL → Sandbox → SDK → Copilot → User
```

Forbidden:

- silent failures
- partial failures
- inconsistent error propagation

36.10 Error Examples

Example 1 — Invalid Path

Json

```
{
  "code": "PATH_NOT_ALLOWED",
  "category": "PATH_ERROR",
  "details": "Path resolves outside allowed directories."
}
```

Example 2 — Permission Missing

Json

```
{
  "code": "PERMISSION_REQUIRED",
  "category": "PERMISSION_ERROR"
}
```

Example 3 — Sandbox Termination

Json

```
{
  "code": "SCPL_VIOLATION",
  "category": "SANDBOX_ERROR",
  "details": "Attempted forbidden system call."
}
```

36.11 Formal Error Invariants

Invariant 1 — Deterministic Codes

```
∀ identical inputs → identical error codes
```

Invariant 2 — No Leakage

```
error.details ∉ sensitive_data
```

Invariant 3 — Mandatory Logging

```
∀ errors → audit_log(entry)
```

Invariant 4 — Category Completeness

```
∀ errors → category ∈ defined_categories
```

Invariant 5 — Non-Recoverable Isolation

```
SANDBOX_ERROR ∨ SYSTEM_ERROR → S6
```

36.12 Summary

Appendix Y defines the deterministic, sanitized, formally verifiable error model.
Errors are categorized, non-leaking, fully logged, and strictly bound to recovery rules.
No raw OS errors, no nondeterministic behavior, no ambiguity.

---

37. Appendix Z — Audit Log Architecture

(TPM-Sealed, Hash-Chained, Non-Repudiable)

This appendix defines the complete audit log architecture of the Local Copilot API.
The audit log is **forensic-grade**, **tamper-evident**, **TPM-sealed**, **hash-chained**, and **non-repudiable**.

No operation can bypass logging.
No entry can be modified or deleted.
No log can be exported without explicit user approval.

37.1 Audit Log Objectives

The audit log must:

- record every operation
- provide non-repudiation
- detect tampering
- support forensic analysis
- integrate with TPM
- remain local-only
- be deterministic
- be append-only

No silent logging.
No partial logging.
No background logging.

37.2 Audit Log Structure

Each entry has the following structure:
Json

```
{
  "timestamp": "ISO-8601",
  "operation": "string",
  "parameters": { ... sanitized ... },
  "result": "success" | "error",
  "error_code": "string | null",
  "sandbox_id": "string",
  "request_id": "string",
  "previous_hash": "hex",
  "entry_hash": "hex",
  "tpm_seal": "TPM_SEALED_BLOB"
}
```

Mandatory fields only.
No optional metadata.

37.3 Hash Chain Architecture

Each entry is linked to the previous one:

```
entry_hash[n] = H(
  timestamp,
  operation,
  parameters,
  result,
  error_code,
  sandbox_id,
  request_id,
  previous_hash[n-1]
)
```

Properties:

- tamper-evident
- non-modifiable
- non-reorderable
- non-deletable

If any entry is altered → chain breaks → log invalid.

37.4 TPM Sealing

Each entry includes a TPM-sealed integrity block:

```
tpm_seal = TPM_Seal(entry_hash, PCR_profile)
```

PCR profile includes:

- Secure Boot
- VBS
- HVCI
- SAC
- baseline integrity hash

If PCRs change → unseal fails → log invalid.

37.5 Log Storage Model

The audit log is stored:

- locally
- encrypted
- append-only
- in fixed-size segments
- with rotation
- with integrity metadata

Rotation rules:

- segment full → seal segment → start new segment
- sealed segments are immutable

37.6 Log Access Rules

Access allowed:

- user
- enterprise admin (local only)

Access forbidden:

- applications
- scripts
- remote systems
- cloud services
- Copilot itself (unless user requests inspection)

37.7 Log Export Rules

Export requires:

- explicit user approval
- sanitized output
- no raw parameters
- no sensitive data
- no TPM secrets
- no PCR values

Export formats:

- JSON
- CSV

Enterprise may enforce encryption.

37.8 Log Sanitization Rules

Sanitized fields:

- file paths → canonical only
- parameters → no content
- errors → deterministic codes only
- application names → canonical only

Forbidden:

- file contents
- user data
- environment variables
- registry values
- memory addresses
- OS error messages

37.9 Log Integrity Verification

Verification steps:

1. verify hash chain
2. verify TPM seals
3. verify segment boundaries
4. verify deterministic ordering
5. verify no missing entries

If any step fails → log flagged as corrupted.

37.10 Log Bypass Prevention

The following are impossible:

- skipping log entry
- modifying log entry
- deleting log entry
- reordering log entries
- injecting fake entries
- disabling logging
- writing logs outside audit subsystem

Any attempt → sandbox termination.

37.11 Log Entry Generation Flow

```
1. Operation completes
2. Normalize result
3. Construct log entry
4. Compute entry_hash
5. Seal with TPM
6. Append to log
7. Flush to disk
8. Zeroize buffers
```

No asynchronous logging.
No deferred logging.

37.12 Log Categories

Each entry is tagged with:

- FILE_OPERATION
- DIRECTORY_OPERATION
- APPLICATION_OPERATION
- WORKFLOW_OPERATION
- NETWORK_OPERATION
- SYSTEM_OPERATION
- SANDBOX_EVENT

No other categories exist.

37.13 Sandbox Event Logging

The following events are logged:

- sandbox initialization
- sandbox termination
- SCPL violations
- memory violations
- baseline degradation
- forbidden syscall attempts
- permission failures

These entries are mandatory.

37.14 Formal Audit Log Invariants

Invariant 1 — Append-Only

```
∀ entries: no deletion, no modification
```

Invariant 2 — Hash Chain Integrity

```
entry[n].previous_hash = entry[n-1].entry_hash
```

Invariant 3 — TPM Binding

```
TPM_Unseal(entry[n].tpm_seal) = entry[n].entry_hash
```

Invariant 4 — Deterministic Ordering

```
entries sorted by timestamp, strictly increasing
```

Invariant 5 — No Missing Entries

```
∀ n: entry[n] exists → entry[n+1] exists or log_end
```

37.15 Summary

Appendix Z defines the complete audit log architecture.
The log is TPM-sealed, hash-chained, append-only, deterministic, and non-repudiable.
No operation can bypass logging.
No entry can be modified or deleted.
The log is forensic-grade and formally verifiable.

---

38. Appendix AA — Baseline Enforcement Engine

(TPM-Anchored, Non-Degradable, Deterministic)

This appendix defines the **Baseline Enforcement Engine (BEE)**, the subsystem responsible for ensuring that the Local Copilot API only operates under a fully trusted, uncompromised system state.

The baseline is **TPM-anchored**, **non-degradable**, **non-bypassable**, and **deterministic**.

If the baseline is invalid → the API is deactivated.

38.1 Baseline Enforcement Objectives

The BEE must:

- verify system trust anchors
- detect security degradation
- prevent operation under unsafe conditions
- enforce fail-closed behavior
- integrate with TPM
- operate deterministically
- support formal verification

No fallback.
No degraded mode.
No exceptions.

38.2 Baseline Components

The baseline consists of **six mandatory components**:

1. **Secure Boot**
2. **TPM 2.0**
3. **VBS (Virtualization-Based Security)**
4. **HVCI (Hypervisor-Enforced Code Integrity)**
5. **SAC (Smart App Control)**
6. **Unsigned Driver Detection**

All components must be valid.

38.3 Baseline Validation Flow

```
1. Read PCR profile
2. Validate Secure Boot
3. Validate TPM availability
4. Validate VBS
5. Validate HVCI
6. Validate SAC state
7. Validate unsigned driver flag
8. Compute baseline integrity hash
9. Compare with expected hash
```

If any step fails → baseline invalid.

38.4 TPM Anchoring

The baseline integrity hash is sealed in the TPM:

```
baseline_seal = TPM_Seal(baseline_hash, PCR_profile)
```

Properties:

- cannot be forged
- cannot be modified
- cannot be replayed
- cannot be unsealed under degraded PCRs

If PCRs change → unseal fails → API disabled.

38.5 Baseline Integrity Hash

The baseline hash includes:

- Secure Boot state
- VBS state
- HVCI state
- SAC state
- unsigned driver flag
- sandbox runtime integrity hash
- SCPL table hash
- schema registry hash

Hash algorithm: SHA-256 (fixed).

38.6 Non-Degradability Rules

The baseline cannot:

- be partially valid
- be overridden
- be bypassed
- be relaxed
- be replaced
- be downgraded

Any degradation → immediate API deactivation.

38.7 Baseline Enforcement Timing

Baseline is validated:

- at system boot
- at API activation
- at sandbox initialization
- before every operation
- after every critical error
- after every sandbox termination

No skipped checks.

38.8 Baseline Failure Handling

If baseline invalid:

```
→ sandbox termination
→ audit log entry
→ API deactivation
→ user notification
```

Recovery requires:

- full system reboot
- restoration of baseline
- successful TPM unseal
- full sandbox reinitialization

No soft recovery.

38.9 Baseline Drift Detection

The BEE detects:

- new unsigned drivers
- disabled Secure Boot
- disabled VBS
- disabled HVCI
- SAC switched to Off
- firmware changes
- kernel tampering
- hypervisor tampering

Any drift → baseline invalid.

38.10 Baseline Enforcement Engine State Machine

States:

1. **B0 — Unverified**
2. **B1 — Validating**
3. **B2 — Valid**
4. **B3 — Invalid**

Transitions:

```
B0 → B1 → B2
B0 → B1 → B3
B2 → B1 → B3   (drift)
```

No transition from B3 → B2 without reboot.

38.11 Forbidden Behaviors

The BEE must not:

- allow degraded operation
- allow partial validation
- allow fallback modes
- allow user override
- allow policy override
- allow sandbox execution under invalid baseline

Any attempt → sandbox termination.

38.12 Formal Baseline Invariants

Invariant 1 — No Execution Without Valid Baseline

```
baseline_valid = false → sandbox_state ∉ {S3, S4}
```

Invariant 2 — TPM Binding

```
TPM_Unseal(baseline_seal) = baseline_hash
```

Invariant 3 — Non-Degradability

```
baseline_valid = true → all_components_valid = true
```

Invariant 4 — Deterministic Validation

```
same_system_state → same_baseline_hash
```

Invariant 5 — Fail-Closed

```
baseline_invalid → API_disabled
```

38.13 Summary

Appendix AA defines the Baseline Enforcement Engine.
It ensures the Local Copilot API only operates under a fully trusted, TPM-anchored, non-degradable system state.
Any deviation results in immediate shutdown, logging, and non-recoverable deactivation.

---

39. Appendix AB — Security Posture Degradation Handling

(Fail-Closed, Non-Recoverable, Logged)

This appendix defines how the Local Copilot API responds to **any degradation** of the system's security posture.
The system must **fail-closed**, **terminate immediately**, **log the event**, and **remain inactive** until the security posture is restored and verified.

No degraded operation.
No fallback.
No partial trust.

39.1 Objectives

The degradation handling subsystem must:

- detect any reduction in system security
- enforce immediate shutdown
- prevent sandbox execution
- prevent permission evaluation
- prevent system call execution
- log the event in the audit log
- require full baseline restoration before reactivation

No exceptions.

39.2 What Constitutes Security Posture Degradation

Security posture is considered degraded if **any** of the following conditions occur:

1. Secure Boot disabled

2. TPM unavailable or PCRs changed

3. VBS disabled

4. HVCI disabled

5. SAC switched to Off

6. Unsigned driver loaded

7. Kernel tampering detected

8. Hypervisor tampering detected

9. Sandbox integrity hash mismatch

10. SCPL table integrity mismatch

11. Schema registry integrity mismatch

12. Baseline hash mismatch

Any single condition → degradation.

39.3 Detection Timing

Degradation is checked:

- at system boot
- at API activation
- at sandbox initialization
- before every operation
- after every operation
- after every sandbox termination
- after any SYSTEM_ERROR
- after any SANDBOX_ERROR

No skipped checks.

39.4 Degradation Response Flow

If degradation detected:

```
1. Abort current operation
2. Zeroize memory regions
3. Flush audit log buffer
4. Write degradation log entry
5. Terminate sandbox
6. Deactivate API
7. Notify user
```

No recovery without reboot.

39.5 Fail-Closed Behavior

Fail-closed means:

- no operations allowed
- no permissions evaluated
- no system calls executed
- no sandbox initialization
- no SDK commands accepted
- no degraded mode

The system becomes inert until baseline restored.

39.6 Non-Recoverability Rules

After degradation:

- sandbox cannot restart
- API cannot reactivate
- permissions cannot be reused
- audit log cannot be suppressed
- SDK cannot bypass checks

Recovery requires:

- full system reboot
- full baseline validation
- successful TPM unseal
- full sandbox reinitialization

No soft recovery.
No partial recovery.

39.7 Audit Logging Requirements

A degradation event must generate a mandatory audit log entry:
Json

```
{
  "timestamp": "ISO-8601",
  "event": "SECURITY_POSTURE_DEGRADED",
  "component": "string",
  "details": "sanitized",
  "sandbox_id": "string",
  "previous_hash": "hex",
  "entry_hash": "hex",
  "tpm_seal": "TPM_SEALED_BLOB"
}
```

Details must be sanitized:

- no driver names
- no kernel details
- no firmware details
- no PCR values

39.8 Degradation Categories

The system classifies degradation into:

1. **BASELINE_DEGRADATION**
2. **RUNTIME_DEGRADATION**
3. **INTEGRITY_DEGRADATION**
4. **HARDWARE_TRUST_DEGRADATION**

Each category triggers identical fail-closed behavior.

39.9 Degradation Examples

Example 1 — Secure Boot Disabled

```
component: "SecureBoot"
details: "Secure Boot state changed."
```

Example 2 — Unsigned Driver Loaded

```
component: "DriverIntegrity"
details: "Unsigned driver detected."
```

Example 3 — TPM PCR Drift

```
component: "TPM"
details: "PCR profile mismatch."
```

39.10 Forbidden Behaviors

The system must not:

- continue operation under degraded posture
- allow user override
- allow enterprise override
- allow partial trust
- allow degraded sandbox initialization
- allow degraded permission evaluation

Any attempt → immediate termination.

39.11 Formal Degradation Invariants

Invariant 1 — No Execution Under Degradation

```
security_posture = degraded → sandbox_state ∉ {S3, S4}
```

Invariant 2 — Mandatory Logging

```
∀ degradation_events → audit_log(entry)
```

Invariant 3 — Non-Recoverable

```
security_posture = degraded → API_active = false
```

Invariant 4 — TPM Binding

```
TPM_Unseal(baseline_seal) = baseline_hash
```

Invariant 5 — Deterministic Shutdown

```
same_degradation_event → same_shutdown_sequence
```

39.12 Summary

Appendix AB defines the complete degradation handling model.
Any reduction in system security results in immediate fail-closed shutdown, mandatory logging, and non-recoverable deactivation until the baseline is restored and verified.

---

40. Appendix AC — SAC Integration Model

(Application Trust, Launch Rules, Enforcement)

This appendix defines how the Local Copilot API integrates with **Smart App Control (SAC)**.
SAC is a mandatory trust anchor.
If SAC is Off or in Evaluation Mode → the Local Copilot API is disabled.

The integration is **strict**, **non-bypassable**, **deterministic**, and **formally verifiable**.

40.1 SAC Integration Objectives

The SAC integration must:

- ensure only trusted applications can be launched
- enforce Microsoft's application trust model
- prevent execution of unsigned or untrusted binaries
- prevent privilege escalation
- prevent script execution
- prevent automation of untrusted UI
- provide deterministic trust decisions
- support baseline enforcement

No fallback.
No overrides.
No degraded mode.

40.2 SAC Trust States

SAC has exactly **three** states:

1. **On**
2. **Evaluation**
3. **Off**

The Local Copilot API requires:

```
SAC = On
```

If SAC = Evaluation or Off → API disabled.

40.3 SAC Enforcement Points

SAC is enforced at:

1. **API activation**
2. **sandbox initialization**
3. **before every app.launch**
4. **before every app.close**
5. **before every app.focus**
6. **after every SYSTEM_ERROR**
7. **after every SANDBOX_ERROR**

No skipped checks.

40.4 Application Trust Model

An application is considered **trusted** if:

- it is signed
- signature is valid
- signature chain is trusted
- binary is unmodified
- binary is not blocked by SAC
- binary is not a script host
- binary is not elevated
- binary is not a system binary
- binary is not a driver installer

If any condition fails → application untrusted.

40.5 Allowed Application Types

Allowed:

- user-installed signed applications
- Microsoft Store apps
- Microsoft-signed desktop apps
- OEM-signed utilities (non-elevated)

Forbidden:

- unsigned apps
- script hosts (PowerShell, CMD, WScript, CScript)
- installers (MSI, setup.exe)
- elevated apps
- system utilities
- debuggers
- interpreters
- compilers
- shells

40.6 SAC-Driven Launch Rules

The operation:

```
app.launch
```

is allowed only if:

1. SAC = On
2. application is trusted
3. application is non-elevated
4. application is non-script
5. application is non-system
6. application is non-installer
7. application is non-debugger

If any rule fails → deterministic error:

```
APPLICATION_NOT_TRUSTED
```

40.7 SAC Enforcement Pipeline

```
1. Resolve application identity
2. Query SAC trust decision
3. Validate signature
4. Validate trust chain
5. Validate non-elevated state
6. Validate non-script classification
7. Validate non-system classification
8. Validate non-installer classification
9. Approve or deny
```

All steps are mandatory.

40.8 SAC Identity Resolution

Identity resolution includes:

- canonical path
- signature metadata
- publisher
- file hash
- trust chain
- SAC classification

Forbidden:

- environment variables
- symlinks
- UNC paths
- NT paths

40.9 SAC Trust Cache

SAC maintains an internal trust cache.

Rules:

- cache is read-only
- cannot be modified by API
- cannot be modified by SDK
- cannot be modified by user
- cannot be bypassed

If cache entry invalid → application untrusted.

40.10 SAC Drift Detection

Drift occurs if:

- SAC switched to Off
- SAC switched to Evaluation
- SAC policy changed
- SAC trust cache invalidated
- SAC signature database updated
- SAC kernel component tampered

Any drift → baseline invalid → API disabled.

40.11 SAC Enforcement in Sandbox

The sandbox enforces:

- no launching untrusted apps
- no interacting with untrusted windows
- no focusing untrusted windows
- no closing untrusted windows

If attempted → SANDBOX_ERROR:

```
APPLICATION_NOT_TRUSTED
```

40.12 SAC Enforcement in SDK

SDK enforces:

- no building commands for untrusted apps
- no building commands for script hosts
- no building commands for installers
- no building commands for elevated apps

If attempted → VALIDATION_ERROR.

40.13 SAC Enforcement in Audit Log

Every app launch attempt logs:

- application identity
- trust decision
- SAC state
- signature validity
- result

No silent failures.

40.14 Formal SAC Invariants

Invariant 1 — SAC Required

```
SAC = On → API_active = true
SAC ≠ On → API_active = false
```

Invariant 2 — No Launch Without Trust

```
trusted(app) = false → app.launch forbidden
```

Invariant 3 — No Script Execution

```
app ∈ script_hosts → forbidden
```

Invariant 4 — No Elevation

```
app.elevated = true → forbidden
```

Invariant 5 — Deterministic Trust Decisions

```
same_app, same_state → same_trust_result
```

40.15 Summary

Appendix AC defines the complete SAC integration model.
SAC is a mandatory trust anchor.
Only trusted, signed, non-elevated, non-script applications may be launched.
Any deviation results in deterministic failure, logging, and potential API deactivation.

---

41. Appendix AD — Network Subsystem

(Static Domains, HTTPS-Only, Deterministic Responses)

This appendix defines the **optional** network subsystem of the Local Copilot API.
Network access is **disabled by default** and only enabled through explicit user permission.
When enabled, the subsystem operates under strict, deterministic, non-bypassable constraints.

No dynamic networking.
No sockets.
No DNS.
No local network.
No cloud sync.
No telemetry.

41.1 Network Subsystem Objectives

The subsystem must:

- allow safe, deterministic HTTP(S) requests
- restrict communication to static, predefined domains
- enforce HTTPS-only
- block dynamic URLs
- block redirects
- block cookies
- block streaming
- sanitize responses
- enforce size limits
- prevent exfiltration
- support formal verification

No exceptions.

41.2 Network Capability Overview

Network operations are limited to:

```
network.request
```

This operation is optional and requires:

- explicit user permission
- static domain
- static path
- HTTPS
- text-only response

41.3 Static Domain Model

Domains must be:

- predefined
- immutable
- hardcoded
- verified
- pinned

Examples (illustrative):

- `api.example.com`
- `data.example.org`

Forbidden:

- user-provided domains
- dynamic domains
- IP literals
- localhost
- LAN addresses
- VPN addresses
- Tor/onion addresses

41.4 URL Structure Rules

Allowed URL structure:

```
https://<static-domain>/<static-path>
```

Constraints:

- domain must be in static allowlist
- path must be static (no variables)
- no query parameters
- no fragments
- no dynamic construction

Forbidden:

- `http://`
- `ftp://`
- `file://`
- `ws://`
- `wss://`
- `data:`
- `blob:`

41.5 Request Method Rules

Allowed:

- GET
- POST

Forbidden:

- PUT
- DELETE
- PATCH
- HEAD
- OPTIONS
- TRACE

41.6 Request Body Rules

POST payload must be:

- UTF-8 text
- ≤ 64 KB
- deterministic
- sanitized

Forbidden:

- binary data
- encoded files
- base64 blobs
- JSON with dynamic keys
- user identifiers

41.7 Response Normalization

Responses must be:

- text-only
- UTF-8
- ≤ 1 MB
- stripped of headers
- stripped of cookies
- stripped of metadata
- stripped of redirects
- stripped of content-type variance

Forbidden:

- HTML with active content
- JavaScript
- CSS
- binary data
- compressed data
- chunked responses
- streaming responses

41.8 Redirect Handling

All redirects are blocked.

If server returns:

- 301
- 302
- 303
- 307
- 308

→ deterministic error:

```
UNSAFE_CONTENT
```

41.9 DNS Handling

DNS is **not** performed by the sandbox.

Rules:

- no DNS resolution
- no dynamic lookup
- no fallback
- no caching

Domains must be resolved by the OS before sandbox invocation.

41.10 Connection Handling

Connections must:

- use TLS 1.2+
- use certificate pinning
- use static cipher suites
- use deterministic handshake
- disallow renegotiation
- disallow compression
- disallow session resumption

Forbidden:

- plaintext
- self-signed certificates
- expired certificates
- wildcard certificates
- mixed content

41.11 Anti-Exfiltration Rules

The subsystem must prevent:

- sending user data
- sending file contents
- sending system information
- sending identifiers
- sending environment variables
- sending telemetry

Only static payloads allowed.

41.12 Network Error Model

Deterministic errors:

- `NETWORK_DISABLED`
- `DOMAIN_NOT_ALLOWED`
- `HTTPS_REQUIRED`
- `RESPONSE_TOO_LARGE`
- `UNSAFE_CONTENT`
- `INVALID_METHOD`
- `INVALID_URL`

No raw HTTP errors exposed.

41.13 Audit Logging

Every network request logs:

- domain
- path
- method
- payload hash
- response size
- error code (if any)
- timestamp
- sandbox ID

No request can bypass logging.

41.14 Formal Network Invariants

Invariant 1 — Static Domains Only

```
domain ∈ static_allowlist
```

Invariant 2 — HTTPS Only

```
scheme = "https"
```

Invariant 3 — No Dynamic URLs

```
url = static_string
```

Invariant 4 — Text-Only Responses

```
response ∈ UTF8_text ∧ size ≤ 1MB
```

Invariant 5 — No Redirects

```
status ∉ {301,302,303,307,308}
```

Invariant 6 — No Exfiltration

```
payload ∉ sensitive_data
```

41.15 Summary

Appendix AD defines the complete network subsystem.
Network access is optional, disabled by default, and strictly constrained to static, HTTPS-only, deterministic requests with sanitized text-only responses.
No dynamic networking, no exfiltration, no redirects, no DNS, no sockets.

---

42. Appendix AE — Deterministic Serialization Protocol

(Canonical Ordering, UTF-8, No Variance)

This appendix defines the **Deterministic Serialization Protocol (DSP)** used by the Local Copilot API SDK and sandbox.
Serialization must be **canonical**, **stable**, **non-ambiguous**, **non-variant**, and **formally verifiable**.

No randomness.
No whitespace variance.
No locale dependence.
No OS dependence.

42.1 Objectives of the Serialization Protocol

The DSP must:

- produce identical byte output for identical input
- enforce canonical key ordering
- enforce UTF-8 encoding
- eliminate formatting variance
- eliminate whitespace variance
- eliminate locale variance
- eliminate nondeterministic fields
- support formal verification
- support deterministic hashing

Serialization is part of the security boundary.

42.2 Canonical Ordering Rules

Keys must be sorted:

- lexicographically
- ascending
- case-sensitive
- stable

Example:
Json

```
{
  "content": "...",
  "operation": "file.create",
  "path": "C:\\Users\\User\\file.txt"
}
```

Forbidden:

- insertion order
- random order
- locale-dependent order
- mixed case ordering rules

42.3 UTF-8 Encoding Rules

All serialized output must:

- be UTF-8
- use no BOM
- use no alternative encodings
- use no normalization variants

Forbidden:

- UTF-16
- UTF-32
- ANSI
- locale-specific encodings

42.4 Whitespace Rules

Whitespace is **fixed**:

- no tabs
- no trailing spaces
- no indentation variance
- no line break variance
- no pretty-printing

Whitespace is minimized and deterministic.

42.5 Numeric Normalization

All numbers must be:

- base-10
- no leading zeros
- no scientific notation
- no locale formatting

Example:

```
42
```

Forbidden:

- `042`
- `4.2e1`
- `4,2`
- `4 2`

42.6 Boolean Normalization

Booleans must be:

```
true
false
```

Lowercase only.
No quotes.

42.7 Null Normalization

Null must be:

```
null
```

Lowercase only.

42.8 String Normalization

Strings must:

- be UTF-8
- escape backslashes
- escape quotes
- escape control characters
- use canonical path formatting

Example:

```
"C:\\\\Users\\\\User\\\\file.txt"
```

Forbidden:

- single quotes
- unescaped backslashes
- unescaped control characters

42.9 Forbidden Fields

The DSP forbids serialization of:

- timestamps (sandbox adds them)
- random IDs (sandbox generates them)
- OS-specific metadata
- environment variables
- locale information
- machine identifiers
- user identifiers

SDK cannot include these fields.

42.10 Deterministic Field Set

Every operation has a **fixed field set** defined in the schema registry.

Forbidden:

- extra fields
- missing fields
- dynamic fields
- metadata injection

If field mismatch → `SCHEMA_MISMATCH`.

42.11 Deterministic Serialization Example

Input:

```
operation = "file.create"
path = "C:\\Users\\User\\file.txt"
content = "Hello"
```

Output:

```
{"content":"Hello","operation":"file.create","path":"C:\\\\Users\\\\User\\\\
file.txt"}
```

Properties:

- sorted keys
- escaped backslashes
- no whitespace variance
- UTF-8
- no BOM

42.12 Serialization Error Conditions

Errors include:

- `INVALID_PARAMETER`
- `SCHEMA_MISMATCH`
- `INVALID_CONTENT`
- `SIZE_LIMIT_EXCEEDED`

If serialization fails → command rejected before IPC.

42.13 Deterministic Hashing

Serialized output is used for:

- audit log hashing
- TPM sealing
- integrity verification

Hash algorithm: SHA-256.

Hash must be:

- stable
- reproducible
- canonical

42.14 Formal Serialization Invariants

Invariant 1 — Canonical Ordering

```
∀ objects: keys(sorted) = keys(serialized)
```

Invariant 2 — UTF-8 Only

```
encoding = UTF8 ∧ BOM = false
```

Invariant 3 — No Variance

```
same_input → same_output
```

Invariant 4 — No Forbidden Fields

```
∀ fields: field ∈ schema
```

Invariant 5 — Deterministic Escaping

```
escape_rules = fixed
```

42.15 Summary

Appendix AE defines the deterministic serialization protocol.
Serialization is canonical, UTF-8, whitespace-stable, and formally verifiable.
No randomness, no locale variance, no OS variance, no dynamic fields.

Appendix Q-Extended — Full Compliance Completion Package

(GDPR, EU Cybersecurity Act, NIST 800-53, ISO 27001, NIS2, BSI, SDL, STRIDE)

This appendix provides the missing compliance components required for a complete, audit-ready specification.
All content is written for enterprise certification, external audit, and regulatory assessment.

---

1. GDPR Compliance Completion

1.1 Mapping of API Functions to GDPR Articles

API Function	GDPR Article	Compliance Mechanism
File operations	Art. 5, 25, 32	Local-only, explicit permission, sandbox isolation
Directory operations	Art. 5, 25	Deterministic access, no metadata leakage
Application operations	Art. 6, 25	Explicit user intent, SAC trust enforcement
Network requests	Art. 44–49	Disabled by default, no cross-border transfer
Audit logging	Art. 30, 32	TPM-sealed, hash-chained, local-only

---

1.2 DPIA (Data Protection Impact Assessment)

- Processing is strictly local
- No cloud transfer
- No identifiers stored
- No profiling
- No automated decision-making
- Risk level: **Low**
- Residual risk: **Minimal** due to sandbox isolation and TPM anchoring

---

1.3 Roles (Controller / Processor)

- **User** = Controller
- **Local Copilot API** = Processor
- **Microsoft** = Not involved (no data leaves device)

1.4 Lawful Basis of Processing

- Art. 6(1)(a) — Explicit consent via permission prompts
- Art. 6(1)(b) — Execution of user-requested operations

1.5 Data Retention & Deletion

- No persistent personal data stored
- Audit logs contain minimal metadata
- Logs stored locally only
- User may delete logs at any time
- No backups, no replication, no cloud sync

1.6 Data Minimization Analysis

- No telemetry
- No identifiers
- No behavioral data
- No content inspection
- Only user-provided paths and parameters processed

1.7 Data Subject Rights Handling

- Access: User can inspect logs
- Rectification: Not applicable (no personal data stored)
- Erasure: Logs deletable
- Portability: Not applicable
- Objection: User can disable API
- Restriction: API inactive without permission

1.8 Technical Measures (Art. 32)

- VBS isolation
- TPM sealing
- SAC enforcement
- Non-executable memory
- Deterministic sandbox
- No network by default
- Hash-chained audit logs

1.9 Audit Logs as Personal Data

Audit logs may contain:

- file paths
- application names

→ Classified as **personal data** under GDPR.
→ Must be protected under Art. 32.

---

2. EU Cybersecurity Act Completion

2.1 Assurance Level Classification

The architecture aligns with:

EUCC High
due to:

- hardware-anchored trust
- deterministic sandbox
- TPM-sealed logs
- strict isolation
- no dynamic code execution

2.2 Mapping to CSA Security Objectives

Objective	Implementation
Protection against known vulnerabilities	SCPL, VBS, HVCI
Secure default configuration	Network off, no scripts
Strong isolation	VBS, sandbox memory model
Transparency	Deterministic behavior, audit logs
Supply-chain trust	SAC signature enforcement

2.3 Supply-Chain Security Requirements

- Only signed binaries allowed
- SAC trust chain validation
- No dynamic loading
- No external dependencies

2.4 Patch Management Requirements

- OS-level patching only
- No API-level patching mechanism
- Sandbox integrity hash updated only via OS update

2.5 Vulnerability Disclosure

- Follows standard enterprise VDP
- No remote attack surface
- Local-only threat model

2.6 Certification Requirements

- EUCC High assessment
- Penetration testing
- Code review
- Supply-chain verification

---

3. NIST 800-53 Completion

3.1 Full Control Mapping

Access Control (AC)

- AC-3: Permission gateway
- AC-6: Least privilege enforced by SCPL
- AC-17: No remote access

Audit & Accountability (AU)

- AU-2: Full operation logging
- AU-6: Audit review supported
- AU-8: Time-stamped, hash-chained logs

System & Communications Protection (SC)

- SC-3: Isolation
- SC-7: Boundary protection
- SC-39: Process isolation

System Integrity (SI)

- SI-3: Malicious code prevention
- SI-7: Integrity checks
- SI-10: Input validation

Identification & Authentication (IA)

- IA-2: OS-level identity
- IA-5: Credential protection (none stored)

Configuration Management (CM)

- CM-2: Baseline enforcement
- CM-6: Configuration restrictions

3.2 Shared Responsibility Matrix

Area	OS	API	Enterprise
Identity	✓	–	–
File access	✓	✓	–
Network	✓	✓	✓
Logging	✓	✓	✓
Baseline	✓	–	–

3.3 Boundary Protection

- No external network
- No inter-process communication except IPC channel
- No shared memory
- No dynamic linking

3.4 Least Privilege Proof

- No elevation
- No privileged syscalls
- No registry modification
- No system directories

3.5 Audit Mechanisms

- AU-2: All operations logged
- AU-6: Logs reviewable
- AU-8: TPM-sealed timestamps

---

4. ISO 27001 Completion

4.1 Annex A Mapping

Control	Implementation
A.5 Policies	Deterministic architecture
A.8 Asset mgmt	No external assets
A.9 Access control	Permission model
A.10 Crypto	TPM sealing
A.12 Ops security	Sandbox isolation
A.13 Communications	Network off
A.14 Acquisition	Formal verification
A.16 Incidents	Logged, fail-closed
A.18 Compliance	GDPR alignment

4.2 Scope Definition

Scope includes:

- Local Copilot API
- Sandbox
- SDK
- Audit subsystem

Scope excludes:

- OS
- Cloud services
- Third-party apps

4.3 Required Controls

Asset Management

- No persistent assets
- Logs only

Access Control

- Explicit permissions
- No elevation

Operations Security

- Deterministic sandbox
- No dynamic code

Cryptographic Controls

- TPM sealing
- SHA-256 hashing

Supplier Management

- SAC trust chain

Compliance

- GDPR, CSA, NIST, ISO

Logging & Monitoring

- TPM-sealed logs
- No remote monitoring

---

5. EU-Wide Additional Requirements

5.1 NIS2 Relevance

- Not a network-exposed service
- Not essential infrastructure
  → **NIS2: Not in scope**

5.2 KRITIS Relevance

- No critical infrastructure function
  → **KRITIS: Not in scope**

5.3 BSI Grundschutz Mapping

- ORP.4 Logging → Fully covered
- SYS.1.2 Clients → Covered via OS
- SYS.1.5 Virtualization → VBS
- APP.4 Application isolation → Sandbox

---

6. Internal Engineering Standards

6.1 SDL Mapping

- Threat modeling: STRIDE
- Static analysis: Required
- Memory safety: Enforced
- No dynamic code: Enforced
- Secure defaults: Enforced

6.2 STRIDE Threat Model

Threat	Mitigation
Spoofing	SAC, TPM
Tampering	Hash chain
Repudiation	TPM-sealed logs
Information disclosure	No telemetry
DoS	Fail-closed
Elevation of privilege	No elevation

6.3 Privacy-by-Design Checklist

- Data minimization: ✓
- Local-only: ✓
- No identifiers: ✓
- Explicit consent: ✓
- No profiling: ✓

6.4 Secure Coding Guidelines

- No pointers
- No dynamic memory
- No recursion
- No JIT
- No unsafe syscalls

6.5 Accessibility (WCAG)

- OS-level UI
- High-contrast support
- Screen reader compatible

---

7. Audit & Certification Requirements

7.1 External Audits

- Annual penetration test
- Annual code review
- EUCC High certification cycle

7.2 Internal Audits

- Quarterly log review
- Baseline integrity checks
- SAC trust chain verification

7.3 Penetration Testing

- Local attack surface only
- No remote vectors
- Sandbox boundary testing

7.4 Red-Team Simulation

- Attempted sandbox escape
- Attempted syscall injection
- Attempted baseline degradation

7.5 Incident Response

- Fail-closed
- Log event
- User notification
- No auto-recovery

---

8. Required Compliance Artifacts

8.1 Data Flow Diagrams

- User → SDK → Sandbox → SCPL → OS
- No external flows

8.2 RACI Matrix

Task	Responsible	Accountable	Consulted	Informed
Baseline	OS	OS	API	User
Logging	API	API	Enterprise	User
Permissions	User	User	API	–

8.3 Compliance Responsibility Matrix

Area	User	API	OS	Enterprise
Data control	✓	–	–	–
Execution	✓	✓	✓	–
Logging	–	✓	–	✓

8.4 Risk Register

- Sandbox escape → mitigated by VBS
- Baseline drift → mitigated by TPM
- Untrusted apps → mitigated by SAC
- Data leakage → mitigated by local-only design

8.5 Control Implementation Evidence

- TPM seals
- Hash chains
- Permission prompts
- Deterministic serialization
- SCPL syscall whitelist

8.6 Change Management

- Versioned schema registry
- Versioned SCPL table
- TPM-anchored integrity
- No dynamic updates

End of Appendix Q-Extended